

OSDP 3.0 Proposed PIV Enhancements

Security Industry Association

Redline: dc80bdf149e8 to dc80bdf149e8 working tree
September 16, 2026

Contents

Changelog	4
1 Introduction	8
2 Changes to OSDP 2.2	9
2.1 Updated Command Definitions	9
2.1.1 Authentication Challenge (osdp_CRAUTH)	9
2.1.2 Response to Challenge (osdp_CRAUTHR)	10
2.1.3 Get PIV Data (osdp_PIVGETDATA)	11
2.1.4 PIV Data Reply (osdp_PIVDATAR)	14
2.1.4 <u>PIV Data Reply (osdp_PIVGETDATAR)</u>	14
2.2 PD-PIV Error Codes for PIV	16
2.2.1 PD-Application-Error-Reply (osdp_PDERROR)	16
2.2.1 <u>PIV Error Reply (osdp_PIVERROR)</u>	17
2.3 Updated Function Codes	17
2.3.1 Function Code 12 – Smart Card Support	17
2.3.2 Function Code 13 – Reader Interfaces	18
2.3.3 Function Code 14 – Biometric Support	18
3 New or Enhanced PIV Commands	19

3.1	PIV Put Data (osdp_PIVPUTDATA)	19
3.2	PIV Application Select (osdp_PIVSELECT)	20
3.3	Card Status (osdp_CARDSTATUS)	21
3.4	Card Status Reply (osdp_CARDSTATUSR)	21
3.5	PIV Secure PIN Entry (osdp_PIVSPE)	23
3.6	PIV Secure PIN Entry Reply (osdp_PIVSPER)	25
3.7	PIV Biometric Match (osdp_PIVBIOMATCH)	27
3.8	PIV Biometric Match Reply (osdp_PIVBIOMATCHR)	28
3.9	TWIC Privacy Key Load (osdp_PIVLOADTPK)	29
3.10	PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)	29
3.11	Set PIV Mode (osdp_PIVMODE)	30
3.12	PIV Auto Status Reply (osdp_PIVSTATUS)	34
3.13	VCI Trust Anchor Load (osdp_PIVVCILOADTA)	36
3.14	VCI Trust Anchor Status (osdp_PIVVCITASTATUS)	40
A	References	42
A.1	Normative References	42
A.2	Informative References	42
B	<u>Sample osdp_PIVDATA Requests</u> <u>PIVGETDATA Worked Examples</u> (Informative)	43
B.1	<u>Reading the exchanges</u>	43
B.2	<u>Source fixtures</u>	44
B.3	<u>Complete Discovery Object</u>	44
B.4	<u>Discovery PIN Usage Policy (0x5F2F)</u>	45
B.5	<u>Read the FASC-N element (0x30)</u>	46
B.6	<u>Read the FASC-N value</u>	47
B.7	<u>Read the UUID element (0x34)</u>	48
B.8	<u>Read the UUID value</u>	49
B.9	<u>Bounded CHUID prefix</u>	50

B.10	<u>Complete certificate object</u>	51
B.11	<u>Read the certificate element (0x70)</u>	52
B.12	<u>Read a compressed certificate element</u>	53
B.13	<u>Read a range within the certificate element</u>	55
B.14	<u>Complete Key History object</u>	56
B.15	<u>BIT Group with no biometric templates</u>	57
B.16	<u>Empty 0x53 container</u>	58
B.17	<u>Zero-byte successful response</u>	59
B.18	<u>Existing child with an empty value</u>	60
B.19	<u>Missing direct child</u>	61
B.20	<u>Element request against an empty response</u>	62
B.21	<u>Ambiguous direct-child selection</u>	63
C	High-Speed Autonomous PIV Data Templates (Informative)	64
C.1	Vector Sources	64
C.2	Supported Profiles	64
C.3	Fixed-Width KMAC Challenge Template	65
C.4	Dynamic Authentication Templates	65
C.4.1	RSA Key Challenge Input	65
C.4.2	ECC Signature Input	66
C.4.3	Worked Dynamic Authentication Examples	67
C.5	APDUs	75
D	Deprecated osdp_GENAUTH	78

Changelog

2026-09-16 PIV command updates:

- Renamed `osdp_PDERROR` to `osdp_PIVERROR` and added `PIVGETDATA` error codes.
- Renamed `PIVDATA/PIVDATAR` to `PIVGETDATA/PIVGETDATAR` and revised encoding, TLV selection, byte ranges, and multi-part replies.
- Clarified use of established credential SM for `GET DATA`, `PUT DATA`, and `SELECT`.

2026-09-02 Made proposal builds self-contained by vendoring the SIL-licensed Carlito typeface, replacing system Calibri and Arial dependencies with real regular, bold, italic, and bold-italic faces, and documenting the XeLaTeX and latexmk requirements. Completed the Enhanced PIV message-name cleanup by applying the compact OSDP naming convention consistently to `osdp_PIVLOADTPK`, `osdp_PIVXMITPAIRING`, `osdp_PIVVCILOADTA`, `osdp_PIVVCITASTATUS`, and `osdp_PIVVCITASTATUSR`; command and reply codes, payloads, and behavior are unchanged. Standardized multi-part messages on the generic OSDP *MpSizeTotal*, *MpOffset*, and *MpFragmentSize* field names and moved the multi-part fields ahead of the re-assembled `osdp_PIVMODE` and `osdp_PIVVCILOADTA` data. The current `osdp_PIVVCILOADTA` definition supersedes the anchor-only multi-part sizing described in the 2026-05-30 entry: its total now includes the 12-byte metadata block. Updated the PIV Auto simulator, generated fixtures, proposal PDF, and redline PDF to exercise and publish the same framing and naming.

2026-08-12 Proposed Secure PIN Entry changes and consistency cleanup:

- Proposed clear, mode-specific SPE behavior for status, PIN acquisition and verification, cached-PIN verification, cache/abort, and attempts-remaining queries. The proposal also defines finite default timeouts, cache and PIN-verified lifecycles, and ACK/poll completion behavior.
- Proposed a one-byte `osdp_PIVSPER` reply whose value is interpreted by the requested mode. The OSDP 2.2 `osdp_PR01SPER` reply remains unchanged.
- Standardized Enhanced PIV names, including renaming the autonomous PIV Auto reply to `osdp_PIVSTATUS`. Corrected tentative code conflicts by assigning `osdp_PIVSELECT` to 0xB0 and `osdp_PDERROR` to 0x8A.

2026-06-18 PIVSELECT, PDERROR, and VCI consistency updates:

- Added `osdp_PIVSELECT` as an OSDP-native direct AID selection command using tentative command code 0xA7; clarified its ISO/IEC 7816 SELECT by DF name mapping, selected-AID status update, and application-state clearing rules.
- Removed stale `osdp_CRAUTHR` `osdp_PDERROR` wording so codes 0x0006/0x0007 retain their common meanings for secure-channel and multi-part errors.
- Clarified that VCI pairing codes are transmitted from the ACU to the PD only over OSDP Secure Channel 2.0 and are used by the PD to establish VCI secure messaging with the credential.
- Tightened and clarified verbiage by replacing imprecise “legacy” terminology throughout normative text with neutral wording such as unchanged, OSDP 2.2-format, and poll-based; retained “legacy” only for withdrawn algorithms such as RSA-1024 test material.

2026-05-30 Added source-backed VCI and PIV Auto examples, clarified PIV Auto interoperability requirements, and improved reviewability:

- Added NIST-derived VCI and PIV APDU fixture coverage, GSA-backed demonstration keys, and reproducible worked examples for PIV Auto and VCI validation. RSA-1024 material is retained only as informative legacy/non-current test-vector material.
- Clarified PIV Auto cryptographic inputs and response handling, including KMAC-derived challenge construction, RSA and ECC GENERAL AUTHENTICATE templates, nonzero UUID requirements, and complete returned 0x7C Dynamic Authentication Templates in `osdp_PIVSTATUS`.
- Aligned related command semantics for capability advertising, `osdp_CARDSTATUS` errors, selective `osdp_PIVDATA` replies, `osdp_PIVSPE` status/abort modes, and VCI trust-anchor inventory.
- Reworked Appendix C and the references section for standards review: examples are explicitly informative, vector provenance is documented, source fixtures are framed as public test material, and normative references now include the needed APDU, KMAC, X.509, and requirement-language specifications.
- Added supporting scripts and tests for deterministic simulation, fixture replay, VCI import/validation, Appendix C generation, YubiKey material loading, live APDU capture, and reproducible PDF/table builds.

2026-05-27 Version renamed from OSDP 2.3 to OSDP 3.0 throughout. Consolidated PIV Auto, biometric-result, VCI trust-anchor, and SPE updates:

- Removed `osdp_PIVSPE` flag 0x01 (retry continuation); each `osdp_PIVSPE` command now initiates exactly one PIN-entry session and `osdp_PIVSPER` terminates the session unconditionally, including on PIN mismatch. ACUs wishing to allow additional attempts **MAY** re-issue `osdp_PIVSPE`, using the *Attempts Remaining* count from the preceding `osdp_PIVSPER` to guide retry policy. Bit 0x01 is now reserved.
- Aligned `osdp_PIVSPER` and `osdp_PIVBIOMATCHR` result codes for credential removal, changed `osdp_PIVBIOMATCHR` 0x03 to PIN verification required, reserved `osdp_PIVBIOMATCHR` 0x07, clarified biometric assurance limits, and specified that only off-card biometric match success returns the 32-byte SHA-256 hash of tag 0xBC.
- Clarified the Function Code 14 biometric-support reference for `osdp_PIVLOADTPK` and added CAC as PIV mode data model 0x03.
- Added PIV Auto KMAC challenge derivation, required series and sequence counters, fixed-width supplemental entropy, and the `osdp_PIVSTATUS` poll reply for reporting credential identity and signed challenge responses. PIV Auto is configured through `osdp_PIVMODE` and is restricted to Secure Channel 2.0 sessions that are not using factory keys.
- Replaced the SP 800-73 Intermediate CVC anchor model with an OSDP-defined VCI trust anchor record (tag 0x7F50) carrying a Profile Identifier (0x80 = 0x01), an eight-byte Issuer Identifier Number (0x42), and a DER-encoded SubjectPublicKeyInfo (0x7F49). Maximum conforming record size is 576 bytes (0x0240), derived from the RSA-4096 key case.

- Added normative IIN derivation rule: the IIN in `osdp_PIVVCILOADTA` and in the trust anchor record **SHALL** be the leftmost eight bytes of the `subjectKeyIdentifier` from the content-signing X.509 certificate.
- Tightened `osdp_PIVVCILOADTA` multi-part encoding: `Anchor Length` and `Mp-SizeTotal` ranges corrected to `0x0000-0x0240`; added normative constraint that `MpOffset + MpFragmentSize` **SHALL NOT** exceed `Anchor Length`; PD **SHALL** verify the reassembled byte string is exactly *Anchor Length* bytes and a single `0x7F50` record before committing.
- Documented variable-length PD storage accounting: PDs **SHALL** account for anchors by their actual *Anchor Length*, not a fixed maximum; replacement is atomic (prior anchor remains if replacement fails); delete releases actual stored bytes.
- Specified the two-tier PD verification path during VCI establishment: direct (trust anchor IIN matches secure messaging CVC IIN, anchor SPKI verifies CVC signature) and intermediate (no direct match triggers retrieval of PIV data object `0x1017/5FC122`; Intermediate CVC Subject Identifier must equal CVC IIN, Role `0x5F4C` must be `0x12`, anchor IIN must match Intermediate CVC IIN, anchor SPKI verifies Intermediate CVC, Intermediate CVC verifies secure messaging CVC). PDs **SHALL NOT** perform X.509 path validation or revocation for VCI trust anchors.
- Added normative ACU post-VCI validation requirements: after `osdp_CARDSTATUSR` reports VCI established, the ACU **SHALL** obtain the content-signing certificate (from cache or via `osdp_PIVDATA` for `0x1017/inner tag 0x70`), verify SKI matches *VCI Anchor IIN*, verify SPKI matches the stored anchor record, verify PIV or PIV-I content-signing ECU OID, verify validity period (expired certificate **SHALL** cause session rejection and anchor unload), and perform X.509 path validation and revocation per deployment policy.
- Restored *VCI Trust Anchor ID* to `osdp_CARDSTATUSR` with normative language; added *VCI CVC IIN* and *VCI Anchor IIN* fields (each 8 bytes) so the ACU receives the IIN pair needed to distinguish direct and intermediate verification paths.
- Updated `osdp_PIVVCITASTATUSR`: clarified that *Available Bytes* reflects variable-length accounting and that the per-record IIN is the anchor selector from the stored trust anchor record, not necessarily the secure messaging CVC IIN reported in `osdp_CARDSTATUSR`.
- Added VCI security preconditions: all VCI-related commands **SHALL** be rejected unless OSDP Secure Channel 2.0 is active and not using factory/default keys; failures reported via `osdp_PDERROR 0x0006`.

2026-01-21 Error-handling cleanup and code assignments: (1) `osdp_PIVPUTDATA` now **SHALL** report failures via `osdp_PDERROR` (status word optional) so writes can't silently succeed. (2) `osdp_PIVSPE` adds result code `0x09` in `osdp_PIVSPER` for "VCI not established." (3) `PDERROR` is credential/global only; PD capability gaps stay `osdp_NAK`. (4) OSDP 2.2-format replies (e.g., `osdp_CRAUTHR`, `osdp_PIVDATAR`) remain unchanged, with optional follow-up `osdp_PDERROR` detail if the PD has it. (5) Documented tentative command/reply codes for the new PIV set: commands `0xA6` (VCI trust-anchor status) through `0xAF` (VCI trust-anchor load), and replies `0x85-0x89` for the new responses.

2026-01-07 Added a PIV Mode interface-mask table aligned to Function Code 13 bits, clarified `osdp_CRAUTHR` reply structure (full GENERAL AUTHENTICATE response without SW1|SW2; status

words reported via `osdp_PDERROR` only), tightened `osdp_CARDSTATUS/osdp_CARDSTATUSR` behavior (solicited-only after PD preparation; error on credential removal), remapped the PIV error codes to the `0x10xx` block per subcommittee guidance, added PIV APDU references, aligned `osdp_PIVDATA` tag selection to allow Discovery (`0x7E`)/BIT (`0x7F61`) subtags in line with SP 800-73-4, clarified that `osdp_PDERROR` conveys a single error per reply, and clarified `osdp_PIVXMITPAIRING` handling so pairing codes can be provided before VCI is established when the credential requires them.

2025-12-10 Consolidated application-level errors on `osdp_PDERROR` (two-byte codes, optional SW1|SW2 for APDU failures, multi-error support); moved the PIV error set to `0x0100-0x01FF` and updated PIV commands to use `osdp_PDERROR` (transport faults remain `osdp_NAK`); made the Global Interface Mask in `osdp_PIVMODE` a required byte (no TLV tag) with an updated field layout; clarified that `osdp_CARDSTATUSR` is solicited-only after PD preparation with poll-based `osdp_RAW` as the aggregated card-present indication; and noted the deprecation of `osdp_GENAUTH` with a recommendation to migrate to `osdp_CRAUTH`.

2025-12-04 Clarified that `osdp_CRAUTHR` returns the full GENERAL AUTHENTICATE data including the Dynamic Authentication Template (excluding only the ISO/IEC 7816 status word).

2025-11-12 Added PIV Biometric Match request/reply and tables (one-bit type values only); moved Enhanced PIV NAKs to a `0x20+` base (added `0x27` Credential Not Present/Removed, `0x28` Biometric Type Not Supported; clarified `0x23` Credential Security Status Not Satisfied); aligned `CRAUTHR` error handling to reference the enhanced NAK set; relocated Enhanced PIV (`0x04`) and PIV Auto (`0x08`) indicators to Function Code 12 and cleaned Function Code 13 to interface-only bits (removed non-interface flags; removed PIV Auto from Function Code 14); updated `CARDSTATUS` gating to Function Code 12 bit `0x04`; added a PIV Mode Flags table and clarified PIV Mode AID encoding; enabled PDF TOC hyperlinks and improved table/caption pagination.

2025-10-30 Clarified `CRAUTH` command/reply behaviour (including RSA, AES, and ECC support), overhauled PIV Mode configuration with TLV-based application and interface controls (including PIV Auto), added card status request/reply messages, expanded PIV data retrieval semantics, introduced PIV Put Data, PIV Secure PIN Entry, TWIC privacy-key load, VCI pairing transmit, and VCI trust-anchor load/status commands, and documented the enhanced error-code set (including VCI/PIV Auto conditions).

2025-09-17 Major PIV Mode and SPE revisions.

2025-06-25 Added multi-AID usage and VCI support notes.

2025-05-27 New function code definitions for 3.0 biometric handling.

1 Introduction

This document captures proposed enhancements to the SIA Open Supervised Device Protocol (OSDP) needed to carry FICAM-compliant Personal Identity Verification (PIV) workflows. The material consolidates draft content, reviewer comments, and interoperability feedback collected during the 2025 working sessions and re-expresses it as normative guidance for command authors and device implementers.

The chapters that follow highlight changes required for existing OSDP 2.2 commands, introduce new transactions where no equivalent existed, and provide supplemental examples that reflect current federal credentialing practices. Wherever the proposal diverges from the baseline specification, the text records both the behavioral requirements and the error handling expectations so that access control units (ACUs) and peripheral devices (PDs) can adopt the profile without relying on vendor extensions.

~~Backward compatibility: when a PD encounters a credential-side error on an~~ The key words **SHALL**, **SHALL NOT**, **SHOULD**, **SHOULD NOT**, and **MAY** are interpreted as described in RFC 2119 and RFC 8174 when they appear in capitals.

Compatibility is specified per command. The enhanced PIVGETDATA request is a breaking change; its response retains ~~OSDP 2.2 -format reply (for example osdp_CRAUTHR or osdp_PIVDATAR); it continues to return the unchanged reply shape (often with MpFragmentSize = 0) and may optionally send an osdp_PDERROR with additional detail on a subsequent osdp_POLL. ACUs are expected to tolerate or ignore the optional osdp_PDERROR; OSDP 2.2 ACUs may log the unknown reply code and proceed~~ multi-part framing and reports failures through a mandatory PIVERROR on a later poll. The CRAUTH failure behavior described in Section 2 is unchanged.

2 Changes to OSDP 2.2

This section documents the normative deltas that apply when an OSDP 2.2 implementation adopts the Enhanced PIV profile. Each subsection references the original command definition and records the precise behavioral updates, encoding clarifications, and new error handling rules introduced by this proposal.

2.1 Updated Command Definitions

2.1.1 Authentication Challenge (osdp_CRAUTH)

OSDP 2.2 transports the ISO/IEC 7816-4 GENERALAUTHENTICATE command through `osdp_CRAUTH`. The OSDP 3.0 profile maintains the same framing fields, named `MpSizeTotal`, `MpOffset`, and `MpFragmentSize` as in the generic OSDP multi-part structure, but clarifies how the command payload maps to the PIV operation selected by the algorithm and key reference.

- The *Algorithm* field continues to be sent in the first fragment only. Codes 0x05, 0x06, and 0x07 select RSA private-key operations and preserve backwards compatibility with OSDP 2.2. Codes 0x08, 0x0A, and 0x0C select AES encryption using 128-, 192-, and 256-bit keys respectively. Codes 0x11 and 0x14 identify ECC P-256 and P-384 mechanisms; the selected PIV key reference determines whether the mechanism is used for ECDSA signature generation or ECDH key establishment.
- The *Key* field retains its role as the card's reference identifier. PDs **SHALL NOT** reinterpret or remap the identifier.
- For RSA algorithms (0x05, 0x06, 0x07), the *Challenge* field carries the encoded datum for the RSA private-key operation. The PD forwards the value unmodified; padding and digest encoding are the responsibility of the ACU.
- For AES algorithms (0x08, 0x0A, 0x0C), the *Challenge* field contains the plaintext block(s) to be encrypted. PDs **SHALL** send the data to the credential without padding; ACUs **SHALL** provide data aligned to the block size dictated by the selected key length and card profile.
- For ECC signature operations using key references 0x9A, 0x9C, or 0x9E with algorithms 0x11 or 0x14, the *Challenge* field carries the pre-hashed input to be signed. The PD **SHALL** issue GENERALAUTHENTICATE with command data encoded as 7C Len 82 00 81 Len Challenge; the credential returns the ECDSA signature in tag 0x82. Exponentiation tag 0x85 **SHALL NOT** be used with 0x9A, 0x9C, or 0x9E.
- For ECC key-establishment operations using key reference 0x9D or retired key-management references, algorithms 0x11 and 0x14 use the *Challenge* field to convey the peer public key. The PD **SHALL** issue GENERALAUTHENTICATE with command data encoded as 7C Len 82 00 85 Len (04 || X || Y), where X and Y are the uncompressed coordinates for the selected curve and are left-padded with 0x00 as required. The credential performs elliptic-curve Diffie-Hellman in accordance with SP 800-73.

Table 1: Authentication Challenge (osdp_CRAUTH)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xA5
2	MpSizeTotal	Length of the complete message, least-significant byte first.	0x0000–0xFFFF
2	MpOffset	Offset of this fragment within the complete message, least-significant byte first.	0x0000–0xFFFF
2	MpFragmentSize	Length of the fragment payload, least-significant byte first.	0x0000–0xFFFF
1	Algorithm	Cryptographic mechanism. Supported values include 0x05/0x06/0x07 (RSA private-key operation), 0x08/0x0A/0x0C (AES encryption with 128/192/256-bit keys), and 0x11/0x14 (ECC P-256/P-384; operation depends on the key reference).	0x00–0xFF
1	Reference Identifier	Key reference passed to GENERALAUTHENTICATE.	0x00–0xFF
0-n	Challenge	For RSA algorithms, the encoded datum for the private-key operation. For AES algorithms, the plaintext block(s) to be encrypted, pre-padded by the ACU as required. For ECC 0x9A/0x9C/0x9E, the pre-hashed signature input in tag 0x81; for ECC 0x9D/retired key-management references, the peer public key in tag 0x85.	0x00–0xFF

Deprecated osdp_GENAUTH The osdp_GENAUTH command is deprecated and expected to be removed in a future revision of this specification. Using a PIV credential to decrypt or process arbitrary data is unsafe and unnecessary for credential verification; implementations **SHOULD** use osdp_CRAUTH flows instead.

2.1.2 Response to Challenge (osdp_CRAUTHR)

The reply continues to use the OSDP 2.2 multi-part format. Successful execution yields a response containing the credential's entire GENERALAUTHENTICATE data (minus the ISO/IEC 7816 status word), including the returned Dynamic Authentication Template; PDs **SHALL NOT** strip outer BER-TLV tags or lengths. If the credential reports an error, the PD **SHALL** return the unchanged osdp_CRAUTHR reply (for example

with `MpFragmentSize = 0`) to preserve OSDP 2.2 behaviour. When available, the PD **SHOULD** also return `osdp_PDERRORPIVERROR` on a subsequent `osdp_POLL` with an applicable PIV code—for example 0x1020 (function not supported by credential), 0x1022 (insufficient credential storage), 0x1023 (credential security status not satisfied, including PIN-required conditions), 0x1024 (data object/tag not found), 0x1025 (VCI not established), or 0x1027 (credential not present/removed). Generic parameter errors may continue to use 0x0005 (incorrect parameter). PIN-entry outcomes are reported through command-specific result paths, such as `osdp_PIVSPER`, when secure PIN entry is used. ACUs **SHALL** tolerate an optional `osdp_PDERRORPIVERROR` following that reply; OSDP 2.2 ACUs may log and ignore the unknown reply code.

Reply structure

`osdp_CRAUTHR` **SHALL** return the credential's full ISO/IEC 7816 GENERALAUTHENTICATE response payload, including the Dynamic Authentication Template, framed by the standard OSDP multi-part fields. PDs **SHALL NOT** include SW1|SW2 in `osdp_CRAUTHR`; any status words from the credential **SHALL** be carried only in `osdp_PDERRORPIVERROR` when a card/APDU failure occurs. PDs **SHALL NOT** strip outer BER-TLV tags or lengths from the returned payload.

Table 2: Response to Challenge (`osdp_CRAUTHR`)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x82
2	MpSizeTotal	Length of the complete response, least-significant byte first.	0x0000–0xFFFF
2	MpOffset	Offset of this fragment within the complete response, least-significant byte first.	0x0000–0xFFFF
2	MpFragmentSize	Length of the fragment payload, least-significant byte first.	0x0000–0xFFFF
0-n	Response Data	Signature, encrypted block, or shared-secret material returned by the credential. The credential's BER-TLV structure is preserved exactly as received (status word excluded).	0x00–0xFF

2.1.3 Get PIV Data (`osdp_PIVGETDATA`)

This Enhanced PIV command replaces the OSDP 2.2 defined `osdp_PIVDATA` with a five-byte payload that selected a PIV object and optional element. The original text did not explicitly specify how to pad identifiers shorter than three bytes, how nested tags should be returned, or how to interpret the data offset. request payload. The ACU **SHALL** use this format only with a PD that advertises Enhanced PIV support (Function Code 12 bit 0x04). The PD **SHALL NOT** infer the old request format from the request length. The reply uses the OSDP 3.0 clarifies these behaviors so that ACUs and PDs apply a single normative approach. multi-part layout. Worked exchanges appear in Appendix B.

The ACU **SHALL** transmit the PIV Object Identifier as three bytes encoded most-significant byte first

Table 3: Get PIV Data (osdp_PIVGETDATA)

Size	Name	Meaning	Value
<u>1</u>	<u>CMND</u>	<u>Command identifier.</u>	<u>0xA3</u>
<u>1</u>	<u>Identifier Length</u>	<u>Number of bytes in Object Identifier.</u> When the identifier contains fewer than three significant bytes, the	<u>0x01–0x03</u>
<u>1-3</u>	<u>Object Identifier</u>	<u>Complete BER tag identifying the card data object. Native byte order; no leading padding.</u>	<u>BER tag</u>
<u>1</u>	<u>Tag Length</u>	<u>Number of bytes in Element Tag. Zero selects the complete object and omits Element Tag.</u>	<u>0x00–0x03</u>
<u>0-3</u>	<u>Element Tag</u>	<u>Complete BER tag of a unique direct child of the object's outer container.</u>	<u>BER tag</u>
<u>2</u>	<u>Offset</u>	<u>Starting byte within the selected object or element. Zero identifies its first tag byte. Least-significant byte first.</u>	<u>0x0000–0xFFFF</u>
<u>2</u>	<u>Requested Length</u>	<u>Maximum bytes to return, least-significant byte first. Zero requests all remaining bytes.</u>	<u>0x0000–0xFFFF</u>

Request encoding ~~The ACU SHALL left-pad the value with or, as required, so that the transmitted field is exactly three bytes. The PD provide valid Object Identifier and Element Tag encodings. The PD SHALL pass Object Identifier to the card unmodified and use Element Tag unmodified to select a matching element from the response. Each length field SHALL equal the number of tag bytes that follow it. Identifier Length is one to three; Tag Length is zero to three. Tag Length zero omits Element Tag. The PD SHALL NOT forward those pad bytes to the credential; it reject invalid field lengths, truncated requests, and trailing request bytes with osdp_PIVERROR 0x0005 (invalid parameter).~~

~~The PD SHALL issue the ISO-7816-4 card's GET DATA APDU using only the significant identifier bytes. If VCI secure messaging operation for the requested object. When credential Secure Messaging (SM) is established, the PD SHALL use the appropriate secure channel automatically; no explicit selector is present in the command. The field previously described as "PIV element ID" is renamed PIV Tag. A tag value of directs the PD to return the that SM context. The credential enforces VCI availability and the requested object's access conditions. The PD SHALL complete BER-TLV structure, including the native outer tag (for example card-side response chaining and remove status words and secure-messaging envelopes before selecting response data. A requested OSDP length does not limit the card-side transfer needed to complete that operation.~~

Selection When Tag Length is zero, the PD **SHALL** select the complete object, including its outer tag, length, and value. The PD **SHALL** preserve the original bytes returned by the credential. PIV responses use 0x53 for ordinary objects and native 0x7E (Discovery) and 0x7F61 (BIT Group Template) or the /wrapper used by other PIV objects. Any non-zero value selects a single root-level tag within the object. When the outer tag is or, containers for Discovery and the BIT Group. Other application profiles may define other containers; whole-object selection does not substitute or remove them.

For nonzero Tag Length, the PD **SHALL** parse the outer container and select exactly one matching direct child, including that child's tag, length, and value. The enclosing object's tag and length are not included. The PD **SHALL NOT** search recursively, decompress the value, or re-encode the selected element. The PD **SHALL** inspect the complete child sequence before accepting a unique match. No match produces 0x1024 (data object or tag not found); multiple matches produce 0x1029 (ambiguous element selection); malformed container or child encoding produces 0x102A (malformed credential-object encoding). Element extraction requires definite BER lengths, a single complete outer container, and child boundaries within that container. To select an inner tag, the PD **SHALL** strip that wrapper before returning the selected tag. For other outer tags search the elements directly inside the object's outer container. In BER-TLV encoding, primitive means the tag marks its value as data bytes; constructed means the tag marks its value as a sequence of TLV elements. For PIV objects returned under tag 0x53, the PD **SHALL** return the requested tag if present and treat the contents as TLV elements even though the tag marks them as primitive. Child values are opaque regardless of their constructed bit; the PD **SHALL NOT** reject the request solely because the wrapper is not /. Tags recursively validate them as ASN.1 structures. Whole-object selection does not require child parsing. An empty card response is valid whole-object data; a nonzero Tag Length against an empty response **SHALL** be a single byte in the range -; multi-byte DER tags are not supported and **MUST** be retrieved by issuing a tag value of and parsing the returned object at the ACU. The data offset now supports one or two-byte encodings. PDs produce 0x1024 (data object or tag not found).

Offset and requested length The PD **SHALL** accept either form and determine which was used based on the command length. The offset applies to the selected payload (entire object when the tag is zero, or the chosen tag otherwise). When a credential-side error occurs, PDs apply Offset and Requested Length once, before dividing the data into fragments. Offset zero identifies the first byte of the selected object's or element's tag. The PD **SHALL** start at Offset and return no more than Requested Length bytes. Requested Length zero requests all remaining bytes. If fewer bytes remain, the PD **SHALL** return those bytes. The resulting bytes are the requested data. Offset equal to the selected-data length produces an empty success; Offset beyond that length produces 0x0005 (invalid parameter). The requested data **SHALL NOT** exceed 65,535 bytes. An oversized through-end result produces 0x102B (transfer length exceeded); a bounded range within a larger object is permitted. A nonzero Offset can skip tag and length bytes. The requested range may begin or end within a tag, length, or value. The PD **SHALL** return the unchanged osdp_PIVDATAR reply (for example with those bytes unchanged). The ACU is responsible for interpreting a partial result.

Acceptance and completion The PD **SHALL** acknowledge an accepted operation with MpFragmentSize = 0) to maintain OSDP 2.2 compatibility. If available, PDs osdp_ACK. Acceptance does not indicate successful card access or parameter validation. The PD **MAY** report additional detail via osdp_PDERROR return the result on a subsequent osdp_POLL using code when security prerequisites are not satisfied, when the requested object or tag is absent, or when the

~~credential is not present or is removed; see Subsection 2.2. ACUs: osdp_PIVGETDATAR for success or osdp_PIVERROR for failure. While processing is incomplete, the PD SHALL tolerate an optional answer polls with osdp_PDERRORACK following that reply; OSDP 2.2 ACUs may log and ignore the unknown reply code.~~ Unsupported commands and transport faults retain their OSDP osdp_NAK responses.

~~1 CMND Command identifier. 0xA3 3 PIV Object Identifier Three-byte identifier of the PIV data object per SP-800-73-4, encoded MSB first. Leading zero bytes SHALL be supplied by the ACU when the identifier is shorter than three bytes; the PD SHALL drop those pad bytes before issuing the APDU. 0x00-0xFF¹ 1 PIV Tag Single-byte ASN.1 DER tag of a root-level element. Tag value selects the entire object; non-zero tags select a root-level element and require the PD to strip the outer /wrapper, verify the length, and return only the tagged value. Multi-byte tags are not directly addressable and SHALL be retrieved by requesting tag. 0x00; or 0x01-0xFF 1-2 Data Offset Byte offset within the requested content. Single-byte and two-byte encodings SHALL be accepted; the PD determines the format from the packet length. 0x00-0xFFFF Only one operation may be outstanding at that PD during this read. Until the read completes or is aborted, the ACU SHALL send only osdp_POLL or osdp_ABORT to that PD. The PD SHALL reject a competing command with osdp_NAK 0x09 (unable to process command record) without replacing the active operation. Traffic to other PDs is unaffected. On osdp_ABORT, the PD SHALL discard pending data or errors, cancel further delivery, and return osdp_ACK. It SHALL NOT subsequently report completion for that operation.~~

2.1.4 PIV Data Reply (osdp_PIVDATAR)

The reply structure is unchanged from a field-count perspective, but OSDP 3.0 aligns the semantics with the command refinements above:

~~MpSizeTotal, MpOffset, and MpFragmentSize continue to be encoded LSB first; they now explicitly report the size, fragment base, and fragment length for the content selected by the osdp_PIVDATA request (entire object or chosen tag). The PD SHALL prepare and retain all requested data before sending the first data fragment. The data SHALL come from one credential operation. The PD SHALL NOT combine data from different credential presentations. Once delivery begins, the retained bytes remain unchanged. Card removal before all requested data is retained produces 0x1027 (credential not present or removed); removal afterward does not change the retained data. A card-operation timeout produces 0x0002 (smart-card timeout), insufficient PD resources produce 0x1021 (insufficient PD memory or storage), and unsatisfied credential security conditions produce 0x1023 (credential security status not satisfied). A failure produces one terminal CARDosdp_DATAPIVERROR SHALL convey the exact bytes obtained from the credential after applying any tag stripping specified by the command. No additional framing or padding is permitted.~~, not an empty osdp_PIVGETDATAR followed by optional error detail. Error definitions appear in Subsection 2.2.

2.1.4 PIV Data Reply (osdp_PIVGETDATAR)

Table 4: PIV Data Reply (osdp_PIVGETDATAR)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x80
2	MpSizeTotal	Length in bytes of the complete data object or tagged element, least-significant <u>Total number of requested data bytes across all fragments. Least-significant</u> byte first.	0x0000–0xFFFF
2	MpOffset	Offset in bytes <u>Position</u> of this fragment within the complete payload, least-significant <u>'s first byte within the requested data. Least-significant</u> byte first.	0x0000–0xFFFF
2	MpFragmentSize	Length in bytes of the data carried in this fragment, least-significant <u>Number of Card Data bytes in this reply. Least-significant</u> byte first.	0x0000–0xFFFF
0-n	Card Data	Requested data bytes exactly as returned by the credential. When a tagged element was selected, this sequence contains the complete selected BER-TLV, including tag, length, and value. The bytes returned in this reply, in their original order.	0x00–0xFF

The PD **SHALL** transfer the requested data using the OSDP 3.0 multi-part structure and usage rules. The ACU retrieves each fragment with osdp_POLL. Every reply contains the reply identifier, the three two-byte multi-part fields, and the next portion of the requested data. The multi-part fields use least-significant byte first, in the order shown in the table.

MpSizeTotal **SHALL** specify the number of requested data bytes, not the original object length. Its value **SHALL** remain unchanged throughout the transfer. *MpOffset* identifies the first byte of this fragment within the requested data. The first fragment **SHALL** have *MpOffset* zero, regardless of Offset in the command. Each subsequent fragment **SHALL** start immediately after the preceding fragment. *MpFragmentSize* **SHALL** equal the number of data bytes in that reply.

Fragment boundaries may occur anywhere within the requested data, including within a tag or length field. The PD **SHALL NOT** add or repeat TLV headers for individual fragments. The ACU reconstructs the requested data by joining the fragments in order. Reply identifiers, multi-part fields, and OSDP packet headers and trailers are not part of that data and are not counted in *MpSizeTotal* or *MpFragmentSize*.

For example, a request for 260 bytes starting at Offset 256 produces *MpSizeTotal* 260. With 114 data bytes per reply, the three fragments have *MpOffset* values 0, 114, and 228, and sizes 114, 114, and 32.

For an empty success, the PD **SHALL** return one *osdp_PIVGETDATAR* with all three multi-part fields zero and no data bytes. For a nonempty result, delivery completes when the distinct accepted fragment lengths sum to *MpSizeTotal*. Normal OSDP retransmission rules apply: replaying a reply does not advance the data offset or execute the card operation again. The ACU **SHALL** discard partial data on cancellation or the OSDP 3.0 early-termination indication (zero fragment size and offset at or beyond total). That indication is not successful completion of a nonempty transfer.

2.2 **PD-PIV Error Codes**~~for PIV~~

PD error codes are carried in *osdp_PDERROR*~~PIVERROR~~ using two-byte codes (least-significant byte first). The common PD codes remain available (for example: 0x0001 *SC_ERROR* (smart-card error, with optional SW1|SW2), 0x0002 *SC_TIMEOUT* (smart-card timeout), 0x0004 *TLV_MISSING* (required TLV missing), 0x0005 *INVALID_PARAM* (invalid parameter), 0x0006 *SECURE_CHANNEL_REQUIRED* (secure channel required), 0x0007 *MP_INVALID* (invalid multi-part transfer), 0x0008 *MP_OUT_OF_SEQUENCE*, (multi-part fragment out of sequence), and 0x0009 *MP_OUT_OF_BOUNDS* (multi-part fragment out of bounds)). The following PIV-specific codes occupy the 0x1000-0x10FF range:

Table 5: Enhanced PIV Error Codes

Code	Description
0x1020	Function not supported by credential (PD capability gaps SHALL use <i>osdp_NAK</i> function-not-supported).
0x1021	Insufficient PD memory or storage to complete the requested operation.
0x1022	Insufficient credential memory or storage space.
0x1023	Credential security status not satisfied.
0x1024	Data object or tag not found.
0x1025	VCI not established.
0x1027	Credential not present or removed.
<u>0x1029</u>	<u>Ambiguous element selection: more than one direct child matches the requested tag.</u>
<u>0x102A</u>	<u>Malformed credential-object encoding encountered during element extraction.</u>
<u>0x102B</u>	<u>Selected response exceeds the supported transfer length.</u>

2.2.1 **PD Application Error Reply (osdp_PDERROR)**

The new assignments are tentative. Unlisted codes, including 0x1026 and 0x1028, remain unassigned by this proposal.

PDs-

2.2.1 PIV Error Reply (osdp_PIVERROR)

~~The PD SHALL use osdp_PDERROR only for credential-side or other global application errors. PIVERROR for errors in PIV operations that are not already defined in reported by a command-specific reply. The error code is two bytes, encoded result. Each reply contains one two-byte error code, least-significant byte first. When a For card/APDU failure occurs (for example SC_ERROR or a PIV APDU failure) failures, the PD MAY append the ISO/IEC 7816 status word (available status word in SW1+, SW2) when it is available to the application layer; otherwise the status word is omitted. A single error is reported per osdp_PDERROR reply. Transport-level faults (framing, CRC, sequence) order. Otherwise, it SHALL omit the status word. The originating command defines when the error is returned; PIVGETDATA errors are returned on a later poll. Transport faults continue to use osdp_NAK. This reply replaces the name osdp_PDERROR; its tentative byte value 0x8A and payload structure are unchanged.~~

Table 6: PIV Error Reply (osdp_PIVERROR)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x8A (tentative)
2	Error Code	Two-byte error code, least-significant byte first.	0x0001-0x10FF
0 or 2	Status Word	Optional ISO/IEC 7816 status word for card/APDU failures; present only when the PD can surface it to the application layer.	-

2.3 Updated Function Codes

2.3.1 Function Code 12 – Smart Card Support

Function Code 12 advertises smart-card transport capabilities and profile support at the PD. The compliance byte uses the following bit assignments:

Table 7: Function Code 12 Compliance Flags

Mask	Description
0x01	PD supports transparent reader mode.
0x02	PD supports extended packet mode.
0x04	PD supports the OSDP 3.0 Enhanced PIV profile, including osdp_PIVGETDATA refinements and related commands.
0x08	PD supports OSDP 3.0 autonomous PIV operation (PIV Auto).

Bits not listed are reserved and **SHALL** be transmitted as zero.

2.3.2 Function Code 13 – Reader Interfaces

OSDP 2.2 defined Function Code 13 as a reader count indicator with a compliance byte fixed at 0x00. Enhanced PIV-capable PDs **SHALL** continue to populate the reader count field, but they **MAY** set bits within the compliance byte to advertise the downstream credential technologies they expose. A value of 0x00 remains valid and retains the original OSDP 2.2 meaning of “interface presence unspecified.”

When a PD reports specific interfaces, it **SHALL** encode the compliance byte using the flags in Table 8. The reader count byte continues to report the number of attached credential interfaces.

Table 8: Function Code 13 Compliance Flags

Mask	Description
0x01	PD presents a contact card interface (ISO 7816 or equivalent).
0x02	PD presents a high-frequency contactless credential interface (13.56 MHz NFC).
0x04	PD presents a low-frequency credential interface (approximately 125 kHz prox).
0x08	PD supports Virtual Contact Interface (VCI) secure messaging for applicable modes.
0x10	PD presents a Bluetooth credential interface.
0x20	PD presents a barcode credential interface.
0x40	PD presents an ultra-high-frequency credential interface (UHF/RAIN RFID).

All other bits are reserved for future use and **SHALL** be transmitted as zero.

2.3.3 Function Code 14 – Biometric Support

Function Code 14 remains backward compatible with OSDP 2.2. A compliance value of 0x00, 0x01, 0x02, or 0x03 retains its original meaning and indicates that the PD does not support the OSDP 3.0 biometric command set. Devices that support the enhanced commands **SHALL** set Bit 7 (0x80). When Bit 7 is asserted, the remaining bits of the compliance byte are interpreted as a capability bitfield. Bits not explicitly defined in Table 9 are reserved and **SHALL** be transmitted as zero.

Table 9: Function Code 14 Compliance Flags

Mask	Description
0x80	PD supports the OSDP 3.0 biometric command set.
0x01	PD performs off-card fingerprint comparison using FIPS 201 BIO ANSI INCITS 378 templates.
0x02	PD performs on-card fingerprint comparison using FIPS 201 OCC processes.
0x04	PD performs off-card iris comparison aligned with FIPS 201 BIO guidance.
0x08	PD performs off-card facial comparison using ANSI INCITS 385-2004 images stored on the credential.

3 New or Enhanced PIV Commands

This section introduces commands defined exclusively for the OSDP 3.0 Enhanced PIV profile.

Unless specified otherwise, each multi-part command or reply defined in this section places *MpSizeTotal*, *MpOffset*, and *MpFragmentSize*, in that order, immediately after *CMND*. Each field is a two-byte unsigned integer encoded least-significant byte first. The bytes following *MpFragmentSize* are a fragment of the logical message data: *MpSizeTotal* is the complete logical-data length, *MpOffset* is the zero-based position of the fragment, and *MpFragmentSize* is the exact number of fragment bytes in the current message. The receiver **SHALL** apply the OSDP multi-part sequencing and bounds rules and reassemble the complete logical data before interpreting command-specific fields. Fixed fields shown after the multi-part fields in a packet table occur at their stated position in the reassembled data and are not repeated in every fragment. The separate *osdp_FILETRANSFER* framing remains unchanged.

3.1 PIV Put Data (*osdp_PIVPUTDATA*)

The *osdp_PIVPUTDATA* command (command code 0xA8) instructs the PD to deliver an ISO 7816-4 PUT DATA operation to the credential. The ACU **SHALL** construct the payload as a complete BER-TLV object, including the desired outer tag (0x7E for the Discovery Object, 0x7F61 for a BIT Group Template, 0x5C/0x53 sequences for other data objects, and so on). The PD **SHALL** forward the payload to the credential using CLA = 0x00, INS = 0xDB, P1 = 0x3F, and P2 = 0xFF. If ~~VCI-secure-messaging~~ credential Secure Messaging (SM) is established, the PD **SHALL** ~~select the secure channel automatically~~ use the established SM context and adjust the CLA accordingly. Chained APDUs **SHALL** be generated by the PD when the payload exceeds a single-card APDU.

Table 10: PIV Put Data (*osdp_PIVPUTDATA*)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xA8 (tentative)
2	MpSizeTotal	Total size of the complete PUT DATA payload, least-significant byte first.	0x0000–0xFFFF
2	MpOffset	Offset of this fragment within the complete payload, least-significant byte first.	0x0000–0xFFFF
2	MpFragmentSize	Number of payload bytes carried in this fragment, least-significant byte first.	0x0000–0xFFFF
0-n	Data	BER-TLV payload to be written to the credential. This field already includes the outer object tag selected by the ACU.	0x00–0xFF

Responses A successful write is acknowledged with `osdp_ACK`. If the PD does not implement the command, it **SHALL** return `osdp_NAK` function-not-supported. When a credential-side failure occurs, the PD **SHALL** report the failure—either immediately or on a subsequent `osdp_POLL`—using `osdp_PERRORPIVERROR` and one of the PIV error codes defined in Subsection 2.2: 0x1020 when the function is not supported by the credential, 0x1021 when the PD lacks buffer space to stage the payload, 0x1022 when the credential has insufficient storage, 0x1023 when security conditions are not met, or 0x1027 when the credential is not present or is removed during the operation. The status word is optional.

3.2 PIV Application Select (`osdp_PIVSELECT`)

The `osdp_PIVSELECT` command (command code 0xB0, tentative) instructs the PD to select the application identified by the supplied Application Identifier (AID) on the currently active credential. The command provides an OSDP-native selector for ISO/IEC 7816-4 SELECT by DF name; it is not a raw APDU tunnel and does not expose APDU header bytes to the ACU. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV) before controllers attempt to use this command.

Table 11: PIV Application Select (`osdp_PIVSELECT`)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xB0 (tentative)
1	Selection Flags	Selection controls. All bits are reserved for future use and SHALL be transmitted as zero.	0x00
1	AID Length	Length of the Application Identifier.	0x05–0x10
5-16	AID	Application Identifier to select by ISO/IEC 7816 DF name.	0x00–0xFF

The PD **SHALL** issue SELECT with CLA = 0x00, INS = 0xA4, P1 = 0x04, P2 = 0x00, Lc = AID Length, Data = AID, and Le = 0x00. The PD **SHALL** request normal FCI return data from the credential and **SHALL** use the active credential interface selected by the current PIV mode or by the PD's card-present workflow. If ~~VCI-secure messaging-credential~~ Secure Messaging (SM) is established and remains valid for the selected application, the PD ~~SHALL select the secure channel automatically~~ use the established SM context and adjust the CLA accordingly; otherwise changing applications **SHALL** clear VCI-dependent state, cached pairing codes, and any selected-application metadata derived from the prior application.

The AID Length field **SHALL** be in the ISO/IEC 7816 application identifier range of 5 to 16 bytes. If the payload is malformed, the AID length is outside that range, or any reserved selection flag is nonzero, the PD **SHALL** report `osdp_PERRORPIVERROR` 0x0005 (invalid parameter). A successful selection is acknowledged with `osdp_ACK`; after success, subsequent PIV commands operate against the selected application until the credential is removed, a new `osdp_PIVSELECT` succeeds, or `osdp_PIVMODE` changes the operating context. The PD **SHALL** update the selected AID reported by `osdp_CARDSTATUSR`; if the credential returns an FCI template containing tag 0x4F, that value **SHALL** be reported, otherwise the PD **SHALL** report the requested AID.

If the PD does not implement the command, it **SHALL** return `osdp_NAK` function-not-supported. Credential-side failures **SHALL** be reported using `osdp_PERRORPIVERROR`: 0x1020 when the credential does not support application selection, 0x1023 when security conditions are not satisfied, 0x1024 when the requested AID is not present, or 0x1027 when the credential is not present or is removed during selection. The status word is optional.

3.3 Card Status (`osdp_CARDSTATUS`)

The `osdp_CARDSTATUS` command (command code 0xA9, tentative) requests the current credential context from the PD. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV) before controllers attempt to use this command. The payload selects the status record that should be returned (currently only the standard profile).

Table 12: Card Status Request (`osdp_CARDSTATUS`)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xA9 (tentative)
1	Status Type	Requested status record. 0x01 selects the standard PIV card status profile. Values 0x02-0x7F are reserved and shall elicit <code>osdp_PIVERROR</code> . Values 0x80-0xFF are reserved for private use.	0x00-0xFF

PDs **SHALL** return `osdp_NAK` function-not-supported when the command itself is not implemented. If the requested status type is not supported or the payload is malformed, PDs **SHALL** report the condition via `osdp_PERRORPIVERROR` (for example 0x0005 `incorrect parameter`)(`invalid parameter`) on a subsequent `osdp_POLL`.

3.4 Card Status Reply (`osdp_CARDSTATUSR`)

In response to `osdp_CARDSTATUS`, the PD reports a ready-to-proceed credential state after completing its card-present workflow (power, application selection, and required reads). PDs **SHALL NOT** send `osdp_CARDSTATUSR` unsolicited on `osdp_POLL`. Controllers **SHOULD** continue to use poll-based `osdp_RAW` replies as the single aggregated “card presented and data acquired” indication; PIN usage policy and VCI connectivity **SHALL NOT** be included in unsolicited poll indications. Detailed readiness (credential type, selected AID, cached data, and readiness flags) **SHALL** be returned only in solicited `osdp_CARDSTATUSR`. When VCI is established, the reply reports both the secure messaging CVC IIN and the loaded anchor IIN used by the PD. These values are identical for direct CVC verification and differ when an Intermediate CVC is used. The PD **SHALL** set the *VCI Trust Anchor ID* to the identifier assigned in `osdp_PIVVCILOADTA`; this value **SHALL** correspond to an entry in the `osdp_PIVVCITASTATUSR` inventory whose IIN matches the *VCI Anchor IIN* field, so that the ACU can correlate reported VCI state with the anchor management record.

Table 13: Card Status Reply (osdp_CARDSTATUSR)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x85 (tentative)
1	Detected Credential Type	Current card interface: 0x00 none, 0x01 ISO 7816 contact, 0x02 ISO 14443 contactless, 0x03 other ISO 7816 credential (for example DESFire, Seos), 0x04 low-frequency credential, 0x05 unknown type, 0x06-0x7F reserved, 0x80-0xFF private use.	0x00–0xFF
1	VCI Status	0x00 VCI not established, 0x01 VCI established. Values 0x02-0x7F reserved, 0x80-0xFF private use.	0x00–0xFF
1	VCI Trust Anchor ID	The Trust Anchor ID used by the PD during VCI establishment. The PD SHALL set this field to the identifier assigned in osdp_PIVVCILOADTA; this value SHALL correspond to an entry in the osdp_PIVVCITASTATUSR inventory whose IIN matches the <i>VCI Anchor IIN</i> field. Set to 0x00 when VCI is not established.	0x00–0xFF
8	VCI CVC IIN	Issuer Identification Number from the secure messaging CVC tag 0x42. Set to all zero when VCI is not established.	0x00–0xFF
8	VCI Anchor IIN	Issuer Identification Number of the loaded VCI trust anchor used by the PD. Set to all zero when VCI is not established or no anchor was used.	0x00–0xFF
2	PIN Usage Policy	PIN Usage Policy bytes (tag 0x5F2F) returned verbatim when available. A value of 0xFFFF indicates the policy is not present or has not yet been read.	0x0000–0xFFFF
1	Selected AID Length	Length (0-16) of the selected AID. 0x00 indicates no application is currently selected.	0x00–0x10

Size	Name	Meaning	Value
0-16	Selected AID	Application Identifier read from the credential (for example from the FCI template). Present only when the length is non-zero.	0x00-0xFF

A PD that cannot provide the requested status **SHALL** return `osdp_NAK` when the command is unsupported. For credential-side failures encountered while preparing status, the PD **MAY** omit `osdp_CARDSTATUSR` data and, if available, report details via `osdp_PERRORPIVERROR` on a subsequent `osdp_POLL`. Successful replies include the entire selected AID (when available) and the fixed two-byte PIN Usage Policy field after the PD has completed its preparation. When PIN Usage Policy bytes from the credential's Discovery Object (tag 0x5F2F) are available, the PD **SHALL** report those two bytes verbatim. When no PIV PIN Usage Policy is present or the policy has not yet been read, the PD **SHALL** report 0xFFFF. The VCI fields describe stable readiness (for example whether a secure channel is established) and the IIN evidence the ACU needs for validation; transient negotiation state **SHALL NOT** be reported. If the credential is removed before preparation completes, the PD **MAY** report the condition via `osdp_PERRORPIVERROR` (for example 0x1027 (credential not present or removed)) instead of `osdp_CARDSTATUSR`. `osdp_CARDSTATUSR` is sent only in response to a matching `osdp_CARDSTATUS` request.

3.5 PIV Secure PIN Entry (`osdp_PIVSPE`)

The `osdp_PIVSPE` command (command code 0xAA, tentative) controls Secure PIN Entry for the Enhanced PIV profile. A PD **SHALL** advertise both Function Code 12 bit 0x04 (Enhanced PIV) and Function Code 15 Secure PIN Entry support before an ACU uses this command. The OSDP 2.2 `osdp_XWR Mode-01 SPE` command and `osdp_PR01SPER` reply remain unchanged.

The ACU is responsible for conveying a clear indication that PIN entry is in progress. While the PD is acquiring a PIN, it **SHALL** suppress `osdp_KEYPAD` reports so that no PIN digit is sent to the ACU.

Table 14: PIV Secure PIN Entry (`osdp_PIVSPE`)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAA (tentative)
1	SPE Mode	Secure PIN Entry operating mode. See Table 15.	0x00-0xFF
1	Start Timeout	Seconds allowed to begin key entry; 0x00 selects a finite PD default.	0x00-0xFF
1	Inter-key Timeout	Seconds allowed between keystrokes; 0x00 selects a finite PD default.	0x00-0xFF
1	Flags	Behaviour modifiers. See Table 16.	0x00-0xFF

Size	Name	Meaning	Value
2	Cache Validity	PIN cache validity in seconds (LSB first). Mode 0x04 requires 0x0001-0xFFFF; other modes transmit 0x0000.	0x0000-0xFFFF

Table 15: SPE Mode Values

Value	Description
0x00	Report the persistent SPE status without initiating PIN entry.
0x01	Acquire a PIN and verify it with the credential currently present.
0x02	Execute VCI PIN verification using a cached PIN.
0x03	Execute normal PIN verification using a cached PIN.
0x04	Acquire and cache a PIN.
0x05	Abort active PIN entry and clear the cached PIN.
0x06	Report attempts remaining without initiating PIN verification.
0x07-0x7F	Reserved for future use.
0x80-0xFF	Private use.

Table 16: SPE Flag Values

Mask	Description
0x02	Retain a cached PIN across credential removal until its validity expires or another clearing event occurs. Valid only with Mode 0x04.
0x08	Auto Execute When Credential Is Presented. Valid only with verification Modes 0x01-0x03.
0x01, 0x04, 0x10-0x80	Reserved and SHALL be transmitted as zero.

Modes are enumerated values, not a bit mask. Mode 0x02 requires VCI; Mode 0x03 uses the normal credential interface selected by the active PIV mode. Mode 0x04 requires a *Cache Validity* from 0x0001 through 0xFFFF. All other modes **SHALL** transmit 0x0000 in that field. The PD **SHALL NOT** transmit an entered or cached PIN to the ACU.

A zero *Start Timeout* or *Inter-key Timeout* selects a finite, PD-defined default; neither value permits an infinite timer. The remaining timeout values specify seconds. Fields that do not apply to a selected mode **SHALL** be transmitted as zero.

Only one osdp_PIVSPE operation may be outstanding. Modes 0x01-0x04 each initiate one operation. The PD returns osdp_ACK when it accepts the request, then returns one osdp_PIVSPER in response to a later osdp_POLL. While such an operation is pending, the ACU **SHALL** send only osdp_POLL, Mode 0x05,

or `osdp_ABORT`. Returning `osdp_PIVSPER` ends the operation unconditionally, including after a PIN mismatch.

Mode 0x00 returns `osdp_PIVSPER` directly. It reports only persistent PIN state; it does not report transient active or waiting states. An ACU recovering from an interrupted session first uses Mode 0x05 or `osdp_ABORT`, then queries Mode 0x00. Mode 0x06 is accepted with `osdp_ACK`; the PD reads the present credential without performing PIN verification and returns the count in a subsequent `osdp_PIVSPER`. This query **SHALL NOT** decrement the credential retry counter. Auto Execute does not make the count available without a present credential.

Mode 0x05 is idempotent. It aborts active or partial PIN entry, clears the cached PIN, and completes with `osdp_ACK`; the PD **SHALL NOT** send a later `osdp_PIVSPER` for the canceled operation. `osdp_ABORT` cancels active or partial PIN entry but preserves a cache that was completed before the active operation.

A cached PIN is used, not consumed. It remains available until its validity expires or it is cleared. The PD **SHALL** clear it on Mode 0x05, PD reset, loss of the OSDP secure channel or applicable security context, or a change to the PIV mode or selected application. Credential removal also clears it unless flag 0x02 was supplied with Mode 0x04. PIN-verified status is bound to the current credential, selected application, interface, and credential security context. The PD **SHALL** clear that status on credential removal or reset, application selection or reselection, interface change, PIV mode change, or VCI teardown, and **SHALL NOT** infer it only from an earlier successful reply.

Flag 0x08 arms the verification selected by Mode 0x01, 0x02, or 0x03 to execute when a credential is presented. It **SHALL** be zero for Modes 0x00, 0x04, 0x05, and 0x06. A cached auto-execute workflow first acquires the PIN with Mode 0x04, then selects normal or VCI verification with Mode 0x03 or 0x02 and flag 0x08.

The PD **SHALL** reject an unsupported command with `osdp_NAK 0x03`, an incorrect command length with `osdp_NAK 0x02`, and an invalid mode, reserved bit, or field combination with `osdp_NAK 0x09`. Accepted credential-side outcomes are reported through `osdp_PIVSPER`, not `osdp_PERRORPIVERROR`.

3.6 PIV Secure PIN Entry Reply (`osdp_PIVSPER`)

The `osdp_PIVSPER` reply (reply code 0x86, tentative) contains one mode-dependent *Result* byte. Removing the former third byte affects only this Enhanced PIV reply; the OSDP 2.2 `osdp_PR01SPER` *Tries* byte is unchanged.

Table 17: PIV Secure PIN Entry Reply (osdp_PIVSPER)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x86 (tentative)
1	Result	Mode-dependent SPE status, attempts count, or operation result.	0x00-0xFF

For Mode 0x00, *Result* is the status bitmap in Table 18. For Mode 0x06, it is the attempts value in Table 19. For Modes 0x01-0x04, it is the operation outcome in Table 20.

Table 18: PIVSPE Status Result (Mode 0x00)

Value	Description
0x00	No cached PIN is present and the current credential is not PIN verified.
0x01	A cached PIN is present.
0x02	The current credential and selected application are PIN verified.
0x03	A cached PIN is present and the current credential and selected application are PIN verified.
0x04-0x7F	Reserved for future use.
0x80-0xFF	Private use.

Table 19: PIVSPE Attempts Result (Mode 0x06)

Value	Description
0x00	The credential PIN is blocked.
0x01-0xFE	Number of PIN verification attempts remaining.
0xFF	Attempts remaining are unavailable, including when no credential is present.

Table 20: PIVSPE Result Codes

Value	Description
0x00	Requested operation completed.
0x01	Timeout – PIN entry did not begin within the allotted time.
0x02	Timeout – inter-digit interval exceeded.
0x03	PIN mismatch.
0x04	Credential removed during operation.
0x05	Credential PIN blocked.
0x06	Wrong interface for the requested mode.

Value	Description
0x07	Incorrect parameters.
0x08	Security condition not met.
0x09	VCI not established for the requested mode.
0x0A	No cached PIN is available.
0x0B–0x7F	Reserved for future use.
0x80–0xFF	Private use.

3.7 PIV Biometric Match (osdp_PIVBIOMATCH)

The osdp_PIVBIOMATCH command (command code 0xAB, tentative) instructs the PD to acquire a live sample and compare it to the specified biometric template from the active application. Depending on the *Match Mode*, the comparison occurs either on the credential (on-card match) or within the PD (off-card match).

Table 21: PIV Biometric Match (osdp_PIVBIOMATCH)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAB (tentative)
1	Biometric Type	Selects the template type to compare. See Table 22.	0x00–0xFF
1	Match Mode	0x00 off-card match; 0x01 on-card match.	0x00–0x01

Table 22: Biometric Type Values

Value	Description
0x01	Cardholder fingerprint template.
0x02	Cardholder facial image.
0x04	Cardholder iris template.

Controllers set *Match Mode* to select off-card or on-card comparison. Off-card flows require the PD to retrieve the appropriate template and, where applicable (e.g., TWIC contactless), to have a valid privacy key loaded via osdp_PIVLOADTPK. On-card flows require the credential to support OCC. If the PD does not implement the command, it **SHALL** return osdp_NAK function-not-supported. Credential-side match outcomes (for example PIN verification required or credential removal during operation) **SHALL** be reported in the subsequent osdp_PIVBIOMATCHR result code per Table 24. Conditions that prevent the command from being attempted (for example unsupported credential function, missing VCI when required, or invalid command parameters) **MAY** be reported via osdp_~~PDERROR~~PIVERROR on a subsequent osdp_POLL.

3.8 PIV Biometric Match Reply (osdp_PIVBIOMATCHR)

After osdp_PIVBIOMATCH completes, the PD reports the result as a poll response using reply code 0x87 (tentative).

Table 23: PIV Biometric Match Reply (osdp_PIVBIOMATCHR)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x87 (tentative)
1	Result	Outcome of the biometric match. See Table 24.	0x00–0xFF
0 or 32	Hash	32-byte SHA-256 hash of the biometric template tag 0xBC data field. The PD SHALL include this field only when <i>Result</i> is 0x00 (off-card match success). The PD SHALL omit this field for on-card match success (0x09) and all failure result codes.	0x00–0xFF

When *Result* is 0x00 (off-card match success), the PD **SHALL** include the 32-byte SHA-256 hash of the biometric template tag 0xBC data field in the reply. For on-card match success (*Result* 0x09) and for all failure result codes, the PD **SHALL** omit the hash field; ACUs **SHALL NOT** rely on a hash received alongside any result other than 0x00.

Off-card biometric comparison demonstrates only that the live sample matched template data presented to the PD. Because off-card biometric objects can be copied or duplicated, an off-card biometric match does not by itself prove authenticity of the credential. On-card biometric comparison is a credential-controlled assertion; the biometric object may be attacker controlled and the PD does not receive the template data being matched. To obtain cryptographic proof associated with an on-card biometric success, the ACU **SHALL** perform an additional authentication using an appropriately verified credential key after the successful on-card comparison.

Table 24: PIVBIOMATCH Result Codes

Value	Description
0x00	Biometric match success (off-card match).
0x01	Biometric match timeout (no entry started).
0x02	Biometric match failed (no match).
0x03	PIN verification required.
0x04	Credential removed during operation.
0x05	PIN verification failed.
0x06	PIN verification timeout.

Value	Description
0x07	Reserved for future use.
0x08	Reserved (PD/credential errors are reported via osdp_PIVERROR).
0x09	On-card match success.
0x0A–0x7F	Reserved for future use.
0x80–0xFF	Private use.

3.9 TWIC Privacy Key Load (osdp_PIVLOADTPK)

The osdp_PIVLOADTPK command (command code 0xAC, tentative) instructs the PD to cache a TWIC Privacy Key (TPK) for use with subsequent biometric operations against a TWIC credential. Function Code 14 is the biometric-support compliance byte; before using this command, controllers **SHOULD** verify that Function Code 14 advertises the OSDP 3.0 biometric command set and the relevant biometric modality bits for the intended TWIC operation.

Table 25: TWIC Privacy Key Load (osdp_PIVLOADTPK)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAC (tentative)
1	Cache Timeout	Number of seconds to retain the cached TPK. 0x00 clears any cached key immediately; 0x01–0xFF permit caching for up to 255 seconds.	0x00–0xFF
1	Control Flags	Bit 0 set requests automatic clearing when the credential is removed; all other bits SHALL be zero.	0x00–0xFF
32	TPK	Trusted Processing Key bytes to cache for the next TWIC biometric operation.	0x00–0xFF

Controllers set *Cache Timeout* to define how long (in seconds) the PD may retain the key. A value of 0x00 clears any cached TPK immediately. Values 0x01–0xFF allow caching for up to 255 seconds. Bit 0 of *Control Flags* directs the PD to clear the cached TPK when the credential is removed; all other bits are reserved and **SHALL** be transmitted as zero. If the PD does not support TWIC TPK caching it **SHALL** return osdp_NAK function-not-supported. Invalid payloads (for example reserved bits set) **SHALL** elicit osdp_PIVERROR 0x0005 ([invalid parameter](#)).

3.10 PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)

The osdp_PIVXMITPAIRING command (command code 0xAD, tentative) delivers an eight-digit pairing code to the PD for use during Virtual Contact Interface (VCI) establishment. The pairing code is transmitted

from the ACU to the PD only over OSDP Secure Channel 2.0, and is used by the PD to establish VCI secure messaging with the credential. Controllers **SHALL NOT** issue this command unless a PIV application is currently selected and VCI mode is active.

Table 26: PIV VCI Pairing Code Transmit (osdp_PIVXMITPAIRING)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAD (tentative)
8	Pairing Code	Eight ASCII digits ('0'-'9') used by the PD during VCI establishment for the currently selected PIV application.	0x30–0x39

The pairing code is encoded as eight ASCII digits ('0'-'9'). If no PIV application is selected, the PD **SHALL** return osdp_PERRORPIVERROR 0x0005 ([invalid parameter](#)). Otherwise, the PD **SHALL** cache the pairing code only as an input to VCI establishment for the currently selected PIV application and use it when required by the credential's Discovery Object PIN Usage Policy (for example when bit 4 is set and bit 3 is clear in the first byte). The pairing code is not evidence that VCI secure messaging is already established, and osdp_PIVXMITPAIRING is not transported over VCI secure messaging. When VCI secure messaging is already active or the credential does not require pairing to establish VCI, the PD **MAY** ignore the supplied code after acknowledging receipt. PDs **SHALL** clear any stored pairing code when VCI is established, when VCI is torn down, or when the active PIV mode changes.

VCI security preconditions VCI trust-anchor load/status, VCI pairing-code transmit, VCI establishment, and PIV operations that depend on VCI **SHALL** be accepted only when OSDP Secure Channel 2.0 is active between the ACU and PD. This OSDP Secure Channel protects ACU-to-PD management traffic, including osdp_PIVXMITPAIRING; VCI secure messaging is the separate PD-to-credential secure channel established by the PD. A PD **SHALL** reject VCI-related functionality when operating with factory/default SCBK or other factory keys. Security-precondition failures **SHALL** be reported using osdp_PERRORPIVERROR 0x0006 ([secure channel required](#)) where the command supports osdp_PERRORPIVERROR; otherwise the command's existing failure reporting path applies.

3.11 Set PIV Mode (osdp_PIVMODE)

The osdp_PIVMODE command (command code 0xAE, tentative) defines the operating context for PIV, TWIC, and related credential transactions. OSDP 3.0 replaces the OSDP 2.2 "Application + AID" fields with a configuration payload that includes a required Global Interface Mask byte, PIV Auto challenge parameters when autonomous operation is enabled, and a TLV list of configuration descriptors so controllers can specify multiple application profiles, preferred interfaces, and optional Application Identifiers. PDs **SHALL** advertise support via Function Code 12 bit 0x04 (Enhanced PIV).

Table 27: Set PIV Mode (osdp_PIVMODE)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAE (tentative)
2	MpSizeTotal	Total size of the reassembled configuration data beginning with PIV Mode Flags, least-significant byte first.	0x0008-0xFFFF
2	MpOffset	Offset of this fragment within the reassembled configuration data, least-significant byte first.	0x0000-0xFFFF
2	MpFragmentSize	Number of configuration data bytes following the multi-part fields in this fragment, least-significant byte first.	0x0000-0xFFFF
1	PIV Mode Flags	First byte of the reassembled configuration data; global behaviour flags (Section 3.11).	0x00-0xFF
1	Config Count	Number of configuration entries (1-32).	0x01-0x20
1	Global Interface Mask	Bitmask limiting interfaces available to all configurations (bit 0 contact, bit 1 contactless, bit 3 Bluetooth; bit 2 reserved and set to zero).	0x00-0x0B
4	Series Counter	Conditional field present only when PIV Auto is enabled. Least-significant byte first.	0x00000000-0xFFFFFFFF
4	Sequence Counter	Conditional field present only when PIV Auto is enabled. Least-significant byte first.	0x00000000-0xFFFFFFFF
32	Supplemental Entropy	Conditional fixed-width field present only when PIV Auto is enabled. All zeroes are permitted but not recommended.	32 octets
0-n	TLV Data	Sequence of Configuration Entry TLVs (Tag 0x01) that follow the Global Interface Mask, or follow the conditional PIV Auto fields when PIV Auto is enabled.	-

The *PIV Mode Flags* byte enables global behaviours such as autonomous operation and VCI usage (Table 28). Bits not listed remain reserved and **SHALL** be transmitted as zero.

Table 28: PIV Mode Flags

Mask	Description
0x01	Enable autonomous PIV (PIV Auto). The PD interrogates credentials, establishes VCI when possible, derives a KMAC256 challenge from the configured counters and entropy, and reports the signed response using <code>osdp_PIVSTATUS</code> .
0x02	Permit VCI usage when available.
0x04	Enforce FICAM strict mode (PIV applications only).

Global and per-entry interface masks use the bit positions in Table 29; the Global Interface Mask bounds the interfaces a PD may use when processing all configuration entries.

Table 29: PIV Mode Interface Masks

Mask	Description
0x01	Permit ISO 7816 contact credential interface use.
0x02	Permit ISO 14443 contactless credential interface use.
0x04	Reserved and SHALL be transmitted as zero.
0x08	Permit Bluetooth credential interface use.
0x10–0x80	Reserved and SHALL be transmitted as zero.

The reassembled configuration data begins at *MpOffset* 0x0000 with *PIV Mode Flags*, followed by the configuration count (1-32) and the required Global Interface Mask byte. *MpSizeTotal* counts all configuration data from *PIV Mode Flags* through the final TLV byte. If and only if PIV Auto is enabled in *PIV Mode Flags*, the Global Interface Mask is followed by the Series Counter, Sequence Counter, and fixed 32-byte Supplemental Entropy field before the TLV list. When PIV Auto is not enabled, those three conditional fields are absent and the TLV list begins immediately after the Global Interface Mask. Each configuration appears as a TLV:

The configuration count **SHALL** equal the number of complete Configuration Entry TLVs in the reassembled payload. Consequently, *MpSizeTotal* **SHALL** be at least eight bytes when PIV Auto is disabled and at least 48 bytes when it is enabled, and **SHALL** equal the exact encoded length of the required header, any conditional PIV Auto fields, and all configuration TLVs. Truncated fields, trailing bytes, or a count that does not match the TLV sequence **SHALL** be treated as malformed configuration data.

- **Global Interface Mask (required)** — One-byte mask (no tag) that limits the interfaces the PD may use for all configurations (see Table 29). A value of 0x00 indicates no global restriction; reserved bits **SHALL** be zero.
- **Configuration Entry (Tag 0x01)** — Contains the data-model identifier, a per-entry interface mask, the AID length, and the optional AID:
 - *Data Model* (0x01 PIV, 0x02 TWIC, 0x03 CAC, values 0x04-0x7F reserved, 0x80-0xFF private use). CAC entries use the same interface and AID encoding as PIV and TWIC entries.

- *Interface Mask* — Uses the same bit positions as the global mask (Table 29) and **SHALL NOT** assert interfaces excluded by the Global Interface Mask. A value of 0x00 means “any interface permitted by the global mask.”
- *AID Length (0-16)* and *AID bytes* when provided.

PDs process configuration entries in order, applying the global mask (when present) before honouring each entry's mask. Invalid masks, unknown data-model identifiers, or malformed TLVs **SHALL** result in `osdp_PDERRORPIVEERROR 0x0005` ([invalid parameter](#)). Controllers **SHALL** send the command over a secure channel where required; the PD **SHALL** return `osdp_PDERRORPIVEERROR 0x0006` ([secure channel required](#)) if the necessary security conditions are not satisfied. Multi-part framing errors result in `osdp_PDERRORPIVEERROR 0x0007` ([invalid multi-part transfer](#)). If the PD cannot cache the requested configuration set, it **SHALL** respond with `osdp_PDERRORPIVEERROR 0x1021` ([insufficient PD memory or storage](#)).

When PIV Auto is enabled, PDs interrogate the credential on presentation, establish VCI automatically when possible, derive the PIV Auto challenge, issue the PIV GENERAL AUTHENTICATE command, and report the signed response without additional ACU prompts. If a pairing code is required, the PD signals the state change via a solicited `osdp_CARDSTATUSR` (not via unsolicited poll); the ACU then provides the code using `osdp_PIVXMITPAIRING` over OSDP Secure Channel 2.0, and the PD uses the code to complete VCI establishment.

PIV Auto configuration **SHALL** be accepted only when OSDP Secure Channel 2.0 is active and the channel is not using a factory/default SCBK or other factory key material. A PD **SHALL** reject PIV Auto configuration or execution with `osdp_PDERRORPIVEERROR 0x0006` ([secure channel required](#)) when these preconditions are not met.

When bit 0x01 of *PIV Mode Flags* enables PIV Auto, the configuration **SHALL** specify a *Series Counter*, a *Sequence Counter*, and a 32-byte *Supplemental Entropy* field. The supplemental entropy field is fixed width and **SHALL** always be present; all zeroes are permitted for compatibility but are not recommended. The ACU **SHALL** ensure that a series counter value does not repeat for the same PD and that no sequence counter value is issued more than once within a series. One conforming implementation strategy is to maintain a monotonic series counter that increments at every ACU boot, similar to the SNMP engine boots counter, and a monotonic sequence counter for challenges issued within that boot series.

The PIV Auto challenge **SHALL** be derived from the active Secure Channel 2.0 session key material using two KMAC256 operations. First, the PD derives a 32-byte PIV Auto KDK with key material formed by concatenating S-ENC, S-MAC1, and S-MAC2; empty input data; output length 32 bytes; and customization string OSDP-PIV-AUTO-KDK-v1. Second, the PD derives the challenge digest using the PIV Auto KDK as the KMAC key, the fixed-width template as input data, output length L, and customization string OSDP-PIV-AUTO-CHALLENGE-v1. The fixed-width template **SHALL** contain the series counter, sequence counter, credential FASC-N, credential UUID, challenged PIV key reference, PIV algorithm identifier, and the 32-byte supplemental entropy field. A successful PIV Auto transaction **SHALL** include the credential UUID; if the UUID cannot be read, the PD **SHALL** report PIV Auto failure instead of deriving a success challenge with an all-zero UUID. The output length L is 32 bytes for SHA-256 profiles and 48 bytes for SHA-384 profiles. PIV Auto challenge derivation **SHALL NOT** use static factory keys.

3.12 PIV Auto Status Reply (osdp_PIVSTATUS)

After a PIV Auto card challenge completes, the PD reports the result as a poll response using reply code 0x89 (tentative). The first osdp_POLL response after completion **SHALL** contain osdp_PIVSTATUS; subsequent fragments, when required, **SHALL** use the same reply and advance *MpOffset* until the complete status payload is delivered. This is an autonomous completion reply; there is no corresponding osdp_PIVSTATUS request command.

Table 30: PIV Auto Status Reply (osdp_PIVSTATUS)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x89 (tentative)
2	MpSizeTotal	Total size of the complete PIV Auto status payload, least-significant byte first.	0x0000–0xFFFF
2	MpOffset	Offset of this fragment within the complete status payload, least-significant byte first.	0x0000–0xFFFF
2	MpFragmentSize	Length of the status payload bytes in this fragment, least-significant byte first.	0x0000–0xFFFF
1	Result	PIV Auto result for the challenged credential. See Table 31.	0x00–0xFF
1	Detail	Additional result detail. Reserved as 0x00 when Result is 0x00; may carry SW1 SW2 low byte when Result is 0x06.	0x00–0xFF
4	Series Counter	Series counter value used to derive the challenge, least-significant byte first.	0x00000000–0xFFFFFFFF
4	Sequence Counter	Sequence counter value used to derive the challenge, least-significant byte first.	0x00000000–0xFFFFFFFF
25	FASC-N	FASC-N read from the credential.	200-bit FASC-N
16	UUID	Credential UUID read from the credential. Success replies SHALL carry a nonzero UUID; failure replies may transmit all zeroes when the UUID is unavailable.	16 octets
1	Key Reference	PIV key reference challenged by the PD.	For example 0x9E
1	Algorithm Identifier	PIV GENERAL AUTHENTICATE algorithm identifier.	For example 0x07

Size	Name	Meaning	Value
2	Signed Response Length	Total length of the signed response, least-significant byte first.	0x0000–0xFFFF
0-n	Signed Response Fragment	Contiguous bytes of the signed response carried in this fragment.	Variable

Result and detail codes follow Table 31. When *Result* is 0x00, *Detail* is reserved and **SHALL** be 0x00. When *Result* is not 0x00, the PD **SHALL** set *Signed Response Length* to 0x0000 unless a credential response was received and is useful for ACU diagnostics.

Table 31: PIV Auto Status Result Codes

Value	Description
0x00	Success. <i>Detail</i> is 0x00, UUID is nonzero, and Signed Response carries the complete returned Dynamic Authentication Template.
0x01	Credential not present or removed before PIV Auto completed.
0x02	VCI establishment or required VCI validation failed.
0x03	Required credential data object, FASC-N, UUID, key, or certificate evidence is unavailable.
0x04	Requested PIV key reference or algorithm is not supported for PIV Auto.
0x05	Credential security status is not satisfied, including missing PIN or pairing preconditions.
0x06	Credential GENERAL AUTHENTICATE operation failed; <i>Detail</i> may carry SW1 SW2 when available.
0x07	Signed response verification failed.
0x08	Timeout or PD internal failure before a definitive credential result was available.
0x09–0x7F	Reserved for future standard result codes.
0x80–0xFF	Private use.

The reply reports the exact series and sequence counters used for the challenge, the FASC-N and UUID read from the credential, and the signed response returned by the credential. The ACU uses these fields to verify counter uniqueness, bind the response to the credential identity, and validate the signature and certificate path using ACU trust policy. A success result **SHALL** include a nonzero credential UUID. The PD **SHALL** preserve the credential's signed response bytes as returned by the PIV operation, excluding transport status words such as SW1/SW2.

If the complete signed response does not fit in one OSDP response, the PD **SHALL** fragment `osdp_PIVSTATUS` using the OSDP 2.2 multi-part convention. *MpSizeTotal* is the total status payload size, *MpOffset* is the zero-based offset of the fragment, and *MpFragmentSize* is the number of payload bytes carried in that fragment. The logical status payload begins at offset zero with the fixed fields; each fragment carries the next contiguous bytes of that payload. A fragment boundary may occur within the fixed fields or within the signed response.

3.13 VCI Trust Anchor Load (osdp_PIVVCILOADTA)

The osdp_PIVVCILOADTA command (command code 0xAF, tentative) loads, replaces, or deletes a Virtual Contact Interface (VCI) trust anchor. The ACU **SHALL** validate and authorize each trust anchor under its deployment certificate policy before loading it. Trust anchors must be cached before a PD can verify card certificates during VCI establishment.

Table 32: VCI Trust Anchor Load (osdp_PIVVCILOADTA)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xAF (tentative)
2	MpSizeTotal	Total size of the reassembled operation data, including the 12-byte metadata block and Anchor Data, least-significant byte first.	0x000C–0x024C
2	MpOffset	Offset of this fragment within the reassembled operation data beginning with Operation, least-significant byte first.	0x0000–0xFFFF
2	MpFragmentSize	Number of operation data bytes following the multi-part fields in this fragment, least-significant byte first.	0x0000–0xFFFF
1	Operation	0x00 load/replace anchor; 0x01 delete anchor. Anchor inventory is requested with osdp_PIVVCITASTATUS.	0x00–0x01
1	Trust Anchor ID	Controller-assigned identifier for the anchor (1-128).	0x01–0x80
8	Issuer Identifier Number	For load/replace, the leftmost eight bytes of the subjectKeyIdentifier from the content-signing X.509 certificate and a match for tag 0x42 in the trust anchor record. For delete, this value SHALL match the stored anchor's IIN.	0x00–0xFF
2	Anchor Length	Total size of the variable-length VCI trust anchor record, least-significant byte first. Set to 0x0000 when deleting.	0x0000–0x0240
0-n	Anchor Data	Fragment bytes of the OSDP-defined VCI trust anchor record. Omitted for delete operations.	0x00–0xFF

The reassembled operation data begins at *MpOffset* 0x0000 with a 12-byte metadata block containing *Operation*, *Trust Anchor ID*, *Issuer Identifier Number*, and *Anchor Length*, followed immediately by *Anchor Data*.

These fields may cross fragment boundaries and are interpreted only after the complete logical data has been reassembled. Controllers set *Operation* to 0x00 to load or replace the anchor identified by *Trust Anchor ID*. Set *Operation* to 0x01 with *Anchor Length* 0x0000 and *MpSizeTotal* 0x000C to delete the anchor. For a delete operation, the supplied IIN **SHALL** match the IIN of the stored anchor selected by *Trust Anchor ID*; a mismatch **SHALL** be rejected with `osdp_PDERRORPIVERROR 0x0005 (invalid parameter)`. Inventory is requested with `osdp_PIVVCITASTATUS`, not with `osdp_PIVVCILOADTA`. The *Issuer Identifier Number* (IIN) is an OSDP-defined anchor selector: it **SHALL** be the leftmost eight bytes of the subjectKeyIdentifier from the content-signing X.509 certificate associated with this anchor. For load/replace, this value **SHALL** match the IIN field (tag 0x42) in the VCI trust anchor record.

The *Anchor Length* declares the exact size of the variable-length VCI trust anchor record. For every operation, *MpSizeTotal* **SHALL** equal 0x000C plus *Anchor Length*. For an operation accepted for processing, the sum of all accepted *MpFragmentSize* values **SHALL** equal *MpSizeTotal*, and no fragment **SHALL** extend beyond *MpSizeTotal*. For load operations, the reassembled bytes following the metadata block **SHALL** contain exactly *Anchor Length* bytes. A conforming VCI trust anchor record **SHALL NOT** exceed 576 bytes (0x0240), making the maximum *MpSizeTotal* 0x024C. The PD **SHALL** accept any valid record length from the minimum BER-TLV encoding through 0x0240, subject to supported-key and storage-capacity checks.

PD storage accounting for VCI trust anchors is variable-length. A PD **SHALL NOT** treat every anchor as consuming the maximum 0x0240 bytes unless that is the PD's documented physical allocation model. When reporting capacity, loading, replacing, or deleting anchors, the PD **SHALL** account for stored anchors by their actual *Anchor Length* values and **SHALL** use the same accounting rule for both `osdp_PIVVCITASTATUSR Available Bytes` and `osdp_PIVVCILOADTA` storage acceptance. A replacement operation **SHALL** be evaluated after releasing the storage occupied by the existing anchor with the same *Trust Anchor ID*; if the replacement cannot be completed, the prior anchor **SHALL** remain available. A delete operation **SHALL** release the storage associated with the deleted anchor.

Table 33: VCI Trust Anchor Record Format

Tag	Length	Name	Value
0x7F50	Variable	VCI Trust Anchor Record	OSDP-defined record used to verify a secure messaging CVC or an Intermediate CVC.
0x80	1	Record Profile Identifier	0x01.
0x42	8	Issuer Identifier Number	Anchor selector. This value SHALL match the IIN in the <code>osdp_PIVVCILOADTA</code> metadata.
0x7F49	1-550	SubjectPublicKeyInfo	DER-encoded X.509 SubjectPublicKeyInfo for the public key trusted by the ACU. Supported public-key algorithms are RSA public keys up to 4096 bits and EC public keys for P-256 or P-384.

The VCI trust anchor record is intentionally not a NIST SP 800-73 Intermediate CVC and does not carry X.509 path-validation state. The ACU is responsible for selecting, loading, unloading, and validating the VCI anchors it permits a PD to use before the PD relies on them. The PD uses the record only as a local public-key verifier

selected by IIN. The maximum record size is derived from the largest supported key case: a 550-byte RSA-4096 SubjectPublicKeyInfo plus the fixed fields above and BER-TLV length overhead. The resulting worst case is 573 bytes; this specification rounds the maximum accepted record size to 576 bytes.

Use during VCI establishment When the PD receives the secure messaging CVC from the credential, it **SHALL** first extract the Issuer Identification Number from tag 0x42. If a loaded VCI trust anchor has a matching IIN, the PD **SHALL** use that anchor's SubjectPublicKeyInfo to verify the secure messaging CVC signature. If verification succeeds, the PD may continue VCI establishment using the public key in the secure messaging CVC.

If no loaded VCI trust anchor matches the secure messaging CVC IIN, the PD **SHALL** retrieve the Secure Messaging Certificate Signer data object (~~PIV-data object~~, tag 5FC122, object identifier 0x5FC122, container identifier 0x1017) from the credential and inspect it for one Intermediate CVC (tag 0x7F21). If the object is absent or no Intermediate CVC is present, validation fails. If an Intermediate CVC is present, the PD **SHALL** verify all of the following before continuing:

- The secure messaging CVC IIN (tag 0x42) equals the Subject Identifier (tag 0x5F20) in the Intermediate CVC.
- The Intermediate CVC Role Identifier (tag 0x5F4C) is 0x12.
- A loaded VCI trust anchor exists whose IIN equals the Intermediate CVC IIN (tag 0x42).
- The Intermediate CVC signature verifies using the matching VCI trust anchor public key.
- The secure messaging CVC signature verifies using the public key carried in the Intermediate CVC.

The PD validates only the CVC chain needed for VCI establishment: either the loaded trust anchor directly verifies the secure messaging CVC, or the loaded trust anchor verifies the Intermediate CVC that then verifies the secure messaging CVC. The PD **SHALL NOT** establish VCI unless this CVC validation succeeds. The PD **SHALL NOT** perform full X.509 path validation, validity-period evaluation, certificate-policy evaluation, or revocation processing for VCI trust anchors; those checks are completed by the ACU before loading anchors and again as an acceptance policy after VCI state is reported.

ACU post-VCI validation After receiving osdp_CARDSTATUSR with VCI established, the ACU **SHALL** validate the content-signing certificate corresponding to the reported *VCI Anchor IIN* before treating the VCI session as accepted for the deployment. This validation is an additional ACU acceptance check and does not replace the PD's required CVC validation during VCI establishment. The ACU **SHALL** perform all of the following:

- Obtain the content-signing X.509 certificate. If the ACU does not already possess a cached copy whose subjectKeyIdentifier leftmost-eight-bytes equals the *VCI Anchor IIN*, the ACU **SHALL** use osdp_PIVGETDATA to retrieve the Secure Messaging Certificate Signer data object (~~PIV-data object~~, tag 5FC122, object identifier 0x5FC122, container identifier 0x1017) and extract the certificate from inner tag 0x70.

- Verify that the leftmost eight bytes of the certificate's subjectKeyIdentifier equal the *VCI Anchor IIN* reported in osdp_CARDSTATUSR.
- Verify that the certificate's SubjectPublicKeyInfo matches the SubjectPublicKeyInfo stored in the loaded VCI trust anchor record for that IIN.
- Verify that the certificate includes the PIV content-signing extended key usage OID (2.16.840.1.101.3.6.7) or the PIV-I equivalent (2.16.840.1.101.3.8.7).
- Verify that the current time falls within the certificate's validity period ($\text{notBefore} \leq \text{now} \leq \text{notAfter}$). If the certificate has expired or is not yet valid, the ACU **SHALL** reject the VCI session and **SHALL** unload the corresponding trust anchor.
- Perform X.509 path validation and revocation checking in accordance with the deployment's certificate validation policy.

Table 34: VCI Trust Anchor Examples from NIST Vectors

Vector	Content Signer	Anchor IIN	SPKI	Record
card_2, card_5	Test PIV Content Signer 4 (EC P-384, direct)	B6103792270FBD08	120 bytes	140 bytes
card_3	Test PIV Content Signer 1 (RSA-2048, intermediate)	9BC5E7735F14EC7F	294 bytes	317 bytes
card_4	Test PIV Content Signer 3 (EC P-256, direct)	1905976BDDCA823D	91 bytes	110 bytes
card_16	Test PIV-I Content Signer 1 (RSA-2048, intermediate)	4CC9C7B3A4F1C83E	294 bytes	317 bytes

In the card_16 intermediate example, the secure messaging CVC IIN is 6F8FF48F42B3E22E; the Intermediate CVC Subject Identifier (tag 0x5F20) is also 6F8FF48F42B3E22E; and the Intermediate CVC IIN (tag 0x42) is 4CC9C7B3A4F1C83E, matching the loaded RSA trust anchor.

PDs **SHALL** reassemble the complete operation data before processing it. For a load or replacement, the PD **SHALL** verify that the reassembled *Anchor Data* portion is exactly *Anchor Length* bytes and is a single 0x7F50 record before committing it to storage. A successful operation returns osdp_ACK. Failures return:

- osdp_NAK function-not-supported when the PD does not support trust-anchor caching.
- osdp_~~PDERROR~~PIVERROR 0x0005 (invalid parameter) for malformed metadata (unknown operation, reserved bits, invalid lengths, MpSizeTotal not equal to 0x000C plus *Anchor Length*, or a reassembled Anchor Data length not equal to *Anchor Length*).
- osdp_~~PDERROR~~PIVERROR 0x0007 (invalid multi-part transfer) when multi-part sequencing fails.

- ~~osdp_PDERROR~~osdp_PIVERROR 0x1021 (insufficient PD memory or storage) when the PD lacks storage space for the anchor.

PDs are not required to maintain a real-time clock. PDs may retain either the supplied VCI trust anchor record or an equivalent internal representation so long as the public key and supplied IIN remain available for the verification flow above.

Trust anchors may be stored in volatile or non-volatile memory at the PD's discretion. Controllers should not assume retention beyond the device's documented behaviour.

3.14 VCI Trust Anchor Status (osdp_PIVVCITASTATUS)

The `osdp_PIVVCITASTATUS` command (command code 0xA6, tentative) enumerates cached trust anchors and reports remaining storage capacity. The request carries no payload. The reply uses the standard multi-part fields (`MpSizeTotal`, `MpOffset`, `MpFragmentSize`) so PDs can stream the inventory across one or more fragments using reply code 0x88 (tentative).

Table 35: VCI Trust Anchor Status Request (osdp_PIVVCITASTATUS)

Size	Name	Meaning	Value
1	CMND	Command identifier.	0xA6 (tentative)

Table 36: VCI Trust Anchor Status Reply (osdp_PIVVCITASTATUSR)

Size	Name	Meaning	Value
1	CMND	Reply identifier.	0x88 (tentative)
2	MpSizeTotal	Length of the complete status payload, least-significant byte first.	0x0000-0xFFFF
2	MpOffset	Offset of this fragment within the status payload, least-significant byte first.	0x0000-0xFFFF
2	MpFragmentSize	Length of the data block in this fragment, least-significant byte first.	0x0000-0xFFFF
1	Total Anchor Count	Number of cached trust anchors.	0x00-0xFF
2	Available Bytes	Remaining storage capacity for additional variable-length trust anchor records, least-significant byte first.	0x0000-0xFFFF

Size	Name	Meaning	Value
0-n	Anchor Records	Zero or more 12-byte inventory records. Each record contains: TrustAnchorID (1 byte for load/delete management), anchor selector IIN (8 bytes from the stored trust anchor record, not necessarily the secure CVC IIN), AnchorLength (2 bytes LSB first, maximum 0x0240), and Attributes (1 byte, reserved). Records that span fragments continue in subsequent replies.	-

The PD first reports the total number of anchors and the remaining free bytes. *Available Bytes* is the amount of trust-anchor storage that remains for additional variable-length VCI trust anchor record data after accounting for currently stored anchors by their actual *Anchor Length* values. Each record thereafter lists a trust anchor by management ID, anchor selector IIN, anchor length, and reserved attributes (transmitted as zero). The IIN in this inventory is the IIN from the stored VCI trust anchor record and is not necessarily the secure messaging CVC IIN reported in `osdp_CARDSTATUSR`. Error responses follow the standard pattern: `osdp_NAK` function-not-supported when the command is not implemented, and `osdp_PDERRORP`~~ERROR~~PIVERROR 0x0007 (invalid multi-part transfer) for multi-part errors.

Table 37: Trust Anchor Record Structure

Field	Size	Meaning	Value Range
Trust Anchor ID	1 byte	Identifier assigned by the ACU; reused to replace or delete an anchor.	0x00–0xFF
Issuer Identifier Number (IIN)	8 bytes	Anchor selector supplied in the VCI trust anchor record.	0x00–0xFF
Anchor Length	2 bytes (LSB first)	Total size of the stored variable-length VCI trust anchor record supplied for this anchor.	0x0000–0x0240
Attributes	1 byte	Reserved for future use; transmitted as zero.	0x00

A References

A.1 Normative References

- Security Industry Association, *Open Supervised Device Protocol (OSDP) v2.2*, 2020.
- National Institute of Standards and Technology, *NIST Special Publication 800-73-4, Interfaces for Personal Identity Verification (PIV) — Part 2: PIV Card Application Card Command Interface*, February 2015.
- National Institute of Standards and Technology, *NIST Special Publication 800-73-5, Interfaces for Personal Identity Verification (PIV) — Part 2: PIV Card Application Card Command Interface*, 2024.
- National Institute of Standards and Technology, *NIST Special Publication 800-78-5, Cryptographic Algorithms and Key Sizes for Personal Identity Verification*, 2024.
- National Institute of Standards and Technology, *Federal Information Processing Standards Publication 201-3, Personal Identity Verification (PIV) of Federal Employees and Contractors*, January 2022.
- National Institute of Standards and Technology, *NIST Special Publication 800-185, SHA-3 Derived Functions: cSHAKE, KMAC, TupleHash, and ParallelHash*, December 2016.
- International Organization for Standardization and International Electrotechnical Commission, *ISO/IEC 7816-4, Identification cards — Integrated circuit cards — Part 4: Organization, security and commands for interchange*.
- Internet Engineering Task Force, *RFC 5280, Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile*, May 2008.
- Internet Engineering Task Force, *RFC 2119, Key words for use in RFCs to Indicate Requirement Levels*, March 1997; and *RFC 8174, Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words*, May 2017.

A.2 Informative References

- National Institute of Standards and Technology, *NIST Internal Report 8347, Personal Identity Verification Cards, Version 2*, 2021.
- U.S. General Services Administration, *Federal Identity, Credential, and Access Management (FICAM) Architecture*, Revision 3, 2023.
- Open Physical, *High Speed Autonomous PIV: Concept Paper*, 2024.

B ~~Sample osdp_PIVDATA Requests~~ PIVGETDATA Worked Examples (Informative)

~~This appendix illustrates representative requests that conforming ACUs may issue using the clarified osdp_PIVDATA command. Object identifiers align with NIST SP 800-73-4 Part 2, Table 6. Each example shows the bytes that appear in the OSDP payload after the~~ These examples show the enhanced request defined in Section 2.1.3, the OSDP 3.0 multi-part response defined in Section 2.1.4, and the error reply defined in Section 2.2.1.

The ACU and PD are simulated. Credential data comes from the identified existing card captures; constructed error cases are identified separately. These examples omit OSDP Secure Channel for clarity. CRC detects transmission errors. The exchanges provide neither authentication nor confidentiality. Some source captures used credential-side secure messaging; the examples use the data recovered after that processing.

The files under `worked-examples/pivdata/` contain the complete frame exchanges, expected results, source hashes, and validation instructions. Each example ID also appears in the data file, so the printed example can be matched to its complete exchange. Long responses are shortened in this appendix only.

B.1 Reading the exchanges

Field values use hexadecimal with a ~~CMNDfield~~; the 0x prefix. Packet dumps use space-separated hexadecimal byte pairs under *Packet bytes (hex)*. Two-byte field values are shown as numbers in the request table and in least-significant-byte-first order in the packet bytes.

An accepted PIVGETDATA command receives ACK. The ACU then sends POLL until the PD returns a data fragment or PIVERROR. The first data fragment has offset zero, even when Offset in the command is nonzero. Further polls retrieve the rest of the data in order, until the ACU has received the advertised total. Polls during processing receive ACK. The examples use a maximum frame size of 0x0080 bytes, including header and CRC; this leaves 0x0072 data bytes per PIVGETDATAR fragment when no security block is present.

A whole-object read includes the object's outer tag, length, and value. An inner-tag read includes the selected element's tag, length, and value, and excludes the enclosing object's tag and length. Offset and Requested Length then determine which bytes to return. Offset zero starts at the selected tag; skipping its tag and length returns only value bytes.

The PD divides the requested data across replies. Each TLV header occurs once in the reassembled data. Each reply has its own OSDP framing, reply identifier, and multi-part fields; the ACU joins only the Card Data bytes. Certificate and biometric values retain their original encoding. Selecting the FASC-N and UUID elements individually allows the ACU to read them regardless of their positions within the CHUID.

All exchanges use PD address 0x01 and CRC-16. Each request table gives the field values; the packet bytes below it show how those values are encoded.

Complete machine-readable exchanges include every frame. The fixture manifest records the inputs, expected results, source locations, and hashes. Hex includes the OSDP header and CRC, without an optional leading mark byte. Long replies show their first 0x18 and last 0x10 bytes; ellipsis text is not transmitted.

B.2 Source fixtures

S1: NIST SD33 card02, contact card capture.

S2: NIST SD33 card01, contact card capture.

The 19 examples below are a representative subset of the 37 checked exchanges in the linked JSON file.

B.3 Complete Discovery Object

Read the complete Discovery Object. Tag Length 0x00 keeps its outer tag 0x7E and its length, followed by all its contents.

Example ID: discovery-whole. Source: S1.

Table 38: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0014 returned bytes in 0x01 data reply.

Table 39: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 0F 00 05 A3 01 7E 00 00 00 00</u> <u>00 4A 01</u>	<u>Read request.</u>
<u>PD</u> <u>strips leading zeroes before constructing the ISO-7816-4 APDU.</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 22 00 06 80 14 00 00 00 14 00</u> <u>7E 12 4F 0B A0 00 00 03 08 00 00 10</u> <u>00 01 00 5F 2F 02 68 20 DB F9</u>	<u>Total 0x0014</u> <u>Offset 0x0000</u> <u>Fragment 0x0014</u>

Object identifier, tag, offset. The PD transmits GET-DATA for tag and returns the complete CHUID-BER-TLV so the

B.4 Discovery PIN Usage Policy (0x5F2F)

Read the PIN Usage Policy from the Discovery Object. The reply contains tag 0x5F2F, length 0x02, and the two policy bytes.

Example ID: discovery-pin-policy. Source: S1.

Table 40: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x02</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x5F2F</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0005 returned bytes in 0x01 data reply.

Table 41: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>can-validate-the-</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 01 7E 02 5F 2F 00</u> <u>00 00 00 03 F0</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 13 00 06 80 05 00 00 00 05 00</u> <u>5F 2F 02 68 20 31 67</u>	<u>Total 0x0005</u> <u>Offset 0x0000</u> <u>Fragment 0x0005</u>

B.5 Read the FASC-N element (0x30)

~~Read the FASC-N ,GUID, and expiration fields. Object identifier , tag , offset . Use this request during device discovery to read feature flags (contact /contactless support, optional biometrics) prior to issuing PIV Mode updates. Object identifier , tag , offset . The PD forwards tag with its nested data so the element from the CHUID without depending on its position. The returned element begins with tag 0x30 and length 0x19, followed by the FASC-N.~~

Example ID: chuid-fascn. Source: S1.

Table 42: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x001B returned bytes in 0x01 data reply.

Table 43: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU can parse the application identifier (tag) and PIN Usage Policy (tag). To retrieve only the AID, repeat the command with tag. Object identifier , tag , offset . Large responses for fingerprint or iris templates require multiple osdp_PIVDATAR fragments; the PIVGETDATA	<u>53 01 12 00 05 A3 03 5F C1 02 01 30</u> <u>00 00 00 00 15 89</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 29 00 06 80 1B 00 00 00 1B 00</u> <u>30 19 D6 50 18 58 55 E5 6D C8 12 79</u> <u>45 A1 6C DB 90 6E 78 80 C0 82 86 50</u> <u>18 43 F5 E1 F1</u>	<u>Total 0x001B</u> <u>Offset 0x0000</u> <u>Fragment 0x001B</u>

B.6 Read the FASC-N value

Read just the FASC-N bytes. Offset 0x0002 skips the selected element's 0x30 0x19 header; Requested Length 0x0019 limits the result to the FASC-N value.

Example ID: chuid-fascn-value. Source: S1.

Table 44: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0002</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0019</u>	<u>Maximum bytes to return.</u>

Result: 0x0019 returned bytes in 0x01 data reply.

Table 45: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 30</u> <u>02 00 19 00 96 DD</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 27 00 06 80 19 00 00 00 19 00</u> <u>D6 50 18 58 55 E5 6D C8 12 79 45 A1</u> <u>6C DB 90 6E 78 80 C0 82 86 50 18 43</u> <u>F5 6C 73</u>	<u>Total 0x0019</u> <u>Offset 0x0000</u> <u>Fragment 0x0019</u>

B.7 Read the UUID element (0x34)

Read the UUID (also called the GUID) element from the CHUID. The returned element begins with tag 0x34 and length 0x10, followed by the UUID.

Example ID: chuid-guid. Source: S1.

Table 46: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0012 returned bytes in 0x01 data reply.

Table 47: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 34</u> <u>00 00 00 00 13 00</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 20 00 06 80 12 00 00 00 12 00</u> <u>34 10 45 85 34 70 A4 03 48 44 AE 15</u> <u>C6 49 59 59 FA D5 FA C7</u>	<u>Total 0x0012</u> <u>Offset 0x0000</u> <u>Fragment 0x0012</u>

B.8 Read the UUID value

Read just the UUID bytes. Offset 0x0002 skips the selected element's 0x34 0x10 header; Requested Length 0x0010 limits the result to the UUID value.

Example ID: chuid-guid-value. Source: S1.

Table 48: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0002</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0010</u>	<u>Maximum bytes to return.</u>

Result: 0x0010 returned bytes in 0x01 data reply.

Table 49: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 02 01 34</u> <u>02 00 10 00 08 EE</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 1E 00 06 80 10 00 00 00 10 00</u> <u>45 85 34 70 A4 03 48 44 AE 15 C6 49</u> <u>59 59 FA D5 75 CB</u>	<u>Total 0x0010</u> <u>Offset 0x0000</u> <u>Fragment 0x0010</u>

B.9 Bounded CHUID prefix

Read the first 0x0010 bytes of the CHUID, starting with its outer tag and length. In this capture, the range ends partway through the FASC-N element.

Example ID: chuid-prefix. Source: S1.

Table 50: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC102</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0010</u>	<u>Maximum bytes to return.</u>

Result: 0x0010 returned bytes in 0x01 data reply.

Table 51: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 02 00 00</u> <u>00 10 00 A1 F4</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 1E 00 06 80 10 00 00 00 10 00</u> <u>53 82 07 C8 EE 02 07 C4 30 19 D6 50</u> <u>18 58 55 E5 5A 2E</u>	<u>Total 0x0010</u> <u>Offset 0x0000</u> <u>Fragment 0x0010</u>

B.10 Complete certificate object

Read the complete certificate object, including its outer 0x53 header and all elements inside it. The result is larger than one reply, so the ACU polls for the remaining fragments.

Example ID: certificate-whole. Source: S1.

Table 52: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>MpSizeTotal/0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x066F returned bytes in 0x0F data replies.

Table 53: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 05 00 00</u> <u>00 00 00 93 3F</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 6F 06 00 00 72 00</u> <u>53 82 06 6B 70 82 06 62 30 82 06 5E</u> <u>... 0x58 bytes omitted ...</u> <u>65 73 74 20 43 41 31 33 30 31 06 03</u> <u>55 04 4F 7E</u>	<u>Total 0x066F</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>POLL/reply</u>	<u>Omitted</u>	<u>MpOffset0x0D exchanges;</u> <u>complete frames are in the</u> <u>JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll 0x0F.</u>

Message	Packet bytes (hex)	Notes
PD PIVGETDATAR	53 81 41 00 05 80 6F 06 3C 06 33 00 49 D9 67 65 56 F7 94 17 43 D7 84 02 9F 61 BF D6 D3 99 B6 DC D1 DE 0D 87 F5 F8 1A A9 CD 4D B8 BB D0 E6 6E 4B F5 75 87 9D AB F1 EF 91 D2 1A 71 01 00 FE 00 F7 26	Total 0x066F Offset 0x063C Fragment 0x0033

B.11 Read the certificate element (0x70)

Read element 0x70, including its tag, encoded length, and certificate bytes. This omits the enclosing 0x53 header and the other elements in the object. The 0x70 header appears once in the returned data.

Example ID: certificate-element. Source: S1.

Table 54: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0666 returned bytes in 0x0F data replies.

Table 55: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 12 00 05 A3 03 5F C1 05 01 70 00 00 00 00 39 81	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 80 00 06 80 66 06 00 00 72 00 70 82 06 62 30 82 06 5E 30 82 04 46 ... 0x58 bytes omitted ... 43 41 31 33 30 31 06 03 55 04 03 13 2A 54 2D 25	Total 0x0666 Offset 0x0000 Fragment 0x0072

Message	Packet bytes (hex)	Notes
<u>POLL/MpFragmentSize</u> <u>fields support</u> <u>reconstruction.</u> <u>Object identifier, tag</u> <u>, offset. The reply</u>	<u>Omitted</u>	<u>0x0D exchanges; complete</u> <u>frames are in the JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll 0x0F.</u>
<u>PD returns the</u> <u>complete PIV</u> <u>Application Identifier</u> <u>BER-TLV-nested</u> <u>inside the Discovery</u> <u>Object, including tag,</u> <u>length, and value</u> <u>PIVGETDATAR</u>	<u>53 81 38 00 05 80 66 06 3C 06 2A 00</u> <u>56 F7 94 17 43 D7 84 02 9F 61 BF D6</u> <u>D3 99 B6 DC D1 DE 0D 87 F5 F8 1A A9</u> <u>CD 4D B8 BB D0 E6 6E 4B F5 75 87 9D</u> <u>AB F1 EF 91 D2 1A 8B 7C</u>	<u>Total 0x0666</u> <u>Offset 0x063C</u> <u>Fragment 0x002A</u>

B.12 Read a compressed certificate element

Read element 0x70 from a card that stores a compressed certificate. The PD returns the original tag, length, and compressed bytes. The ACU can read the object's certificate-information element to determine how to interpret the value.

Example ID: certificate-compressed-element. Source: S2. Tags that occupy more than one byte (for example)are outside the scope of this selector and must be obtained by requesting tag.

Table 56: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x05EA returned bytes in 0x0E data replies.

Table 57: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 05 01 70</u> <u>00 00 00 00 39 81</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 EA 05 00 00 72 00</u> <u>70 82 05 E6 1F 8B 08 00 00 00 00 00</u> <u>... 0x58 bytes omitted ...</u> <u>CE F4 47 FF F4 CF 5D BE EF 9C 7B CF</u> <u>39 F7 79 A2</u>	<u>Total 0x05EA</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>POLL/reply</u>	<u>Omitted</u>	<u>0x0C exchanges; complete</u> <u>frames are in the JSON file.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll 0x0E.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 2E 00 07 80 EA 05 CA 05 20 00</u> <u>7F C4 30 BC D9 5C B8 05 0A 09 18 39</u> <u>F8 6D A5 80 16 10 4D DC 8D 38 FC 05</u> <u>BF 05 DC 1B AF 07 00 00 96 99</u>	<u>Total 0x05EA</u> <u>Offset 0x05CA</u> <u>Fragment 0x0020</u>

B.13 Read a range within the certificate element

Read 0x0104 bytes starting at offset 0x0100 of the certificate element, counting from tag 0x70. The reply offsets are 0x0000, 0x0072, and 0x00E4; each identifies a position within the 0x0104 returned bytes.

Example ID: certificate-offset256. Source: S1.

Table 58: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x70</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0100</u>	<u>Bytes to skip from the selected tag.</u>
<u>Requested Length</u>	<u>0x0104</u>	<u>Maximum bytes to return.</u>

Result: 0x0104 returned bytes in 0x03 data replies.

Table 59: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 12 00 05 A3 03 5F C1 05 01 70</u> <u>00 01 04 01 EC 6A</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 06 80 04 01 00 00 72 00</u> <u>14 30 12 06 03 55 04 0B 13 0B 54 65</u> <u>... 0x58 bytes omitted ...</u> <u>00 03 82 01 0F 00 30 82 01 0A 02 82</u> <u>01 01 CE C4</u>	<u>Total 0x0104</u> <u>Offset 0x0000</u> <u>Fragment 0x0072</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 07 60 E9 55</u>	<u>Poll 0x02.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 80 00 07 80 04 01 72 00 72 00</u> <u>00 C3 C3 53 BF 0F 03 48 37 48 A4 BA</u> <u>... 0x58 bytes omitted ...</u> <u>4A A1 F7 74 E7 A2 DE BC F3 87 AA 94</u> <u>76 21 24 7E</u>	<u>Total 0x0104</u> <u>Offset 0x0072</u> <u>Fragment 0x0072</u>

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 05 60 8B 33</u>	<u>Poll 0x03.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 2E 00 05 80 04 01 E4 00 20 00</u> <u>FA 54 AA 2C DF 26 69 45 7E 6D CD 3A</u> <u>48 EE 5E 4B 31 FD A9 4A D8 73 01 A7</u> <u>5C C8 50 D2 0A 60 D1 CC 14 CD</u>	<u>Total 0x0104</u> <u>Offset 0x00E4</u> <u>Fragment 0x0020</u>

B.14 Complete Key History object

Read the complete Key History object. Its outer 0x53 header and all elements inside it are returned unchanged.

Example ID: key-history. Source: S1.

Table 60: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC10C</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x000A returned bytes in 0x01 data reply.

Table 61: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 0C 00 00</u> <u>00 00 00 71 77</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 18 00 06 80 0A 00 00 00 0A 00</u> <u>53 08 C1 01 04 C2 01 00 FE 00 F3 05</u>	<u>Total 0x000A</u> <u>Offset 0x0000</u> <u>Fragment 0x000A</u>

B.15 BIT Group with no biometric templates

Read a Biometric Information Template (BIT) Group with no biometric templates. The reply keeps the outer 0x7F61 tag and the element that reports a count of zero.

Example ID: bit-empty. Source: constructed example.

Table 62: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x02</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7F61</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0006 returned bytes in 0x01 data reply.

Table 63: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 02 7F 61 00 00 00</u> <u>00 00 3E 14</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 14 00 06 80 06 00 00 00 06 00</u> <u>7F 61 03 02 01 00 35 7C</u>	<u>Total 0x0006</u> <u>Offset 0x0000</u> <u>Fragment 0x0006</u>

B.16 Empty 0x53 container

Read an object with no contents. The reply still contains two bytes, 0x53 0x00, because whole-object selection includes the outer tag and length.

Example ID: empty-5300. Source: constructed example.

Table 64: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x03</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x5FC105</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0002 returned bytes in 0x01 data reply.

Table 65: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 11 00 05 A3 03 5F C1 05 00 00</u> <u>00 00 00 93 3F</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 10 00 06 80 02 00 00 00 02 00</u> <u>53 00 5D 38</u>	<u>Total 0x0002</u> <u>Offset 0x0000</u> <u>Fragment 0x0002</u>

B.17 Zero-byte successful response

Handle a successful card response with zero data bytes. The PD sends one PIVGETDATAR with all three multi-part fields zero.

Example ID: empty-success. Source: constructed example.

Table 66: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x00</u>	<u>Select the whole object.</u>
<u>Element Tag</u>	<u>Omitted</u>	<u>Tag Length is zero.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: 0x0000 returned bytes in 0x01 data reply.

Table 67: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 0F 00 05 A3 01 7E 00 00 00 00</u> <u>00 4A 01</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVGETDATAR</u>	<u>53 81 0E 00 06 80 00 00 00 00 00 00</u> <u>C0 05</u>	<u>Total 0x0000</u> <u>Offset 0x0000</u> <u>Fragment 0x0000</u>

B.18 Existing child with an empty value

Read an element whose value is empty. The reply contains its tag and zero length, 0x30 0x00, for a total of two bytes.

Example ID: zero-length-child. Source: constructed example.

Table 68: Request fields

Field	Value	Notes
Command	0xA3	osdp_PIVGETDATA
Identifier Length	0x01	Bytes in Object Identifier.
Object Identifier	0x7E	Object to read.
Tag Length	0x01	Bytes in Element Tag.
Element Tag	0x30	Element to select.
Offset	0x0000	Start at the selected tag.
Requested Length	0x0000	All remaining bytes.

Result: 0x0002 returned bytes in 0x01 data reply.

Table 69: Command and reply exchange

Message	Packet bytes (hex)	Notes
ACU PIVGETDATA	53 01 10 00 05 A3 01 7E 01 30 00 00 00 00 DC 37	Read request.
PD ACK	53 81 08 00 05 40 39 35	Command accepted.
ACU POLL	53 01 08 00 06 60 D8 66	Poll 0x01.
PD PIVGETDATAR	53 81 10 00 06 80 02 00 00 00 02 00 30 00 24 66	Total 0x0002 Offset 0x0000 Fragment 0x0002

B.19 Missing direct child

Request an element that is absent from the object. The PD accepts the command, then reports PIVERROR on the next poll.

Example ID: missing-child. Source: constructed example.

Table 70: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: osdp_PIVERROR 0x1024 (data object or tag not found); see Section 2.2.1.

Table 71: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 34 00 00</u> <u>00 00 DA BE</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVERROR</u>	<u>53 81 0A 00 06 8A 24 10 A3 74</u>	<u>0x1024 (data object or tag not found)</u>

B.20 Element request against an empty response

Request an element when the card returns no data. The PD reports that the tag was not found. A whole-object request for the same response would succeed with zero bytes.

Example ID: empty-missing-child. Source: constructed example.

Table 72: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x34</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: osdp_PIVERROR 0x1024 (data object or tag not found); see Section 2.2.1.

Table 73: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 34 00 00</u> <u>00 00 DA BE</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVERROR</u>	<u>53 81 0A 00 06 8A 24 10 A3 74</u>	<u>0x1024 (data object or tag not found)</u>

B.21 Ambiguous direct-child selection

Request a tag that occurs twice directly inside the outer container. Element selection requires a unique match, so the PD reports an error.

Example ID: duplicate-child. Source: constructed example.

Table 74: Request fields

Field	Value	Notes
<u>Command</u>	<u>0xA3</u>	<u>osdp_PIVGETDATA</u>
<u>Identifier Length</u>	<u>0x01</u>	<u>Bytes in Object Identifier.</u>
<u>Object Identifier</u>	<u>0x7E</u>	<u>Object to read.</u>
<u>Tag Length</u>	<u>0x01</u>	<u>Bytes in Element Tag.</u>
<u>Element Tag</u>	<u>0x30</u>	<u>Element to select.</u>
<u>Offset</u>	<u>0x0000</u>	<u>Start at the selected tag.</u>
<u>Requested Length</u>	<u>0x0000</u>	<u>All remaining bytes.</u>

Result: osdp_PIVERROR 0x1029 (ambiguous element selection); see Section 2.2.1.

Table 75: Command and reply exchange

Message	Packet bytes (hex)	Notes
<u>ACU</u> <u>PIVGETDATA</u>	<u>53 01 10 00 05 A3 01 7E 01 30 00 00</u> <u>00 00 DC 37</u>	<u>Read request.</u>
<u>PD</u> <u>ACK</u>	<u>53 81 08 00 05 40 39 35</u>	<u>Command accepted.</u>
<u>ACU</u> <u>POLL</u>	<u>53 01 08 00 06 60 D8 66</u>	<u>Poll 0x01.</u>
<u>PD</u> <u>PIVERROR</u>	<u>53 81 0A 00 06 8A 29 10 FF 02</u>	<u>0x1029 (ambiguous element selection)</u>

C High-Speed Autonomous PIV Data Templates (Informative)

This appendix records the PIV Auto challenge, ISO/IEC 7816 GENERAL AUTHENTICATE templates, and fixture sources used by the demonstration vectors. The APDU examples are validated by the repository tests against the same helper code used by the simulator and replay tools.

C.1 Vector Sources

Table 76: PIV Auto Vector Sources

Source	Use	Notes
NIST SD 33 / IR 8347	Live-card APDU fixtures	Cards 2, 3, 4, 5, and 16 provide VCI contactless enrolment, OPACITY, and key-challenge transcripts.
NIST SM/VCI captures	CVC and trust anchors	Provides validator-ready 7F21, 5FC122, and 7F50 objects.
GSA FICAM card-builder	Local keys and certificates	Used only as reusable local test material for source-backed simulation.
Generated YubiKey PIV	Missing algorithm profiles	Reusable RSA-1024, RSA-3072, and P-384 test keys are committed as public fixtures and loaded through <code>ykman piv</code> ; RSA-1024 is informative legacy test material only.

C.2 Supported Profiles

Table 77: PIV Auto Supported Profiles

Profile	Alg	Key Ref	PIV Auto	Coverage
RSA-1024	06	9A, 9E	No	Informative legacy/non-current test-only material.
RSA-2048	07	9A, 9E	Yes	NIST SD 33 cards 2, 3, 16; generated-card; GSA-backed simulation.
RSA-3072	05	9A, 9E	Yes	Committed YubiKey material; requires YubiKey firmware with RSA-3072 support.
ECC P-256	11	9A, 9E	Yes	NIST SD 33 cards 4 and 5; generated-card coverage.
ECC P-384	14	9A, 9E	Yes	Committed YubiKey material. NIST card 5 also provides 9C/9D APDU examples.

C.3 Fixed-Width KMAC Challenge Template

Table 78: PIV Auto KMAC Challenge Template

Field	Size	Encoding	Notes
Template label	14	ASCII plus version byte	OSDP-PIV-AUTO followed by 01.
Series Counter	4	LSB first	ACU-managed boot or epoch counter, for example 07000000.
Sequence Counter	4	LSB first	ACU-managed per-series counter, for example 2A000000.
FASC-N	25	Raw bytes	Credential FASC-N from CHUID.
UUID	16	Raw bytes	Credential UUID. Successful PIV Auto examples use the credential UUID; unavailable UUID is a failure condition.
PIV Key Reference	1	Raw byte	Challenged key reference, normally 9A or 9E.
PIV Algorithm	1	Raw byte	Algorithm identifier: 05, 06, 07, 11, or 14.
Supplemental Entropy	32	Raw bytes	Fixed-width ACU entropy field; all zeroes are valid for test vectors only.

The challenge digest is produced in two steps. First, derive PIV-AUTO-KDK=KMAC256(key=S-ENC|S-MAC1|S-MAC2,data=empty,mac_len=32,custom="OSDP-PIV-AUTO-KDK-v1"). Second, derive the challenge as KMAC256(key=PIV-AUTO-KDK,data=fixed-widthtemplate,mac_len=L,custom="OSDP-PIV-AUTO-CHALLENGE-v1"). L is 32 bytes for SHA-256 profiles and 48 bytes for SHA-384 profiles. The supplemental entropy field is always present and fixed at 32 bytes; all zeroes are valid for test vectors but not recommended for deployments.

C.4 Dynamic Authentication Templates

C.4.1 RSA Key Challenge Input

Table 79: RSA Dynamic Authentication Template

Tag / Bytes	Length	Name	Value
7C	Variable	Dynamic Authentication Template	Outer template for the PIV GENERAL AUTHENTICATE command data.
82	00	Response request	Empty value requesting the card response in tag 82.
81	Modulus length	Challenge block	EMSA-PKCS1-v1_5 encoded digest block with the same length as the RSA modulus.
0001FF...FF00	Variable	Padding prefix	EMSA-PKCS1-v1_5 block prefix before the digest information.
30...	Variable	DigestInfo	ASN.1 DigestInfo for SHA-256 or SHA-384 followed by the KMAC-derived digest; SHA-256 uses prefix 3031300D060960864801650304020105000420 and SHA-384 uses prefix 3041300D060960864801650304020205000430.

Table 80: RSA Dynamic Authentication Examples

Profile	Alg	Key Ref	Tag 81	Response 82	Command APDU
RSA-1024	06	9A/9E	128 bytes	128 bytes	Informative legacy/non-current test-only APDU.
RSA-2048	07	9A/9E	256 bytes	256 bytes	274-byte extended APDU.
RSA-3072	05	9A/9E	384 bytes	384 bytes	402-byte extended APDU.

C.4.2 ECC Signature Input

Table 81: ECC Dynamic Authentication Template

Tag / Bytes	Length	Name	Value
7C	Variable	Dynamic Authentication Template	Outer template for the PIV GENERAL AUTHENTICATE command data.
82	00	Response request	Empty value requesting the card response in tag 82.
81	Digest length	Challenge digest	KMAC-derived digest supplied directly to the ECC signing operation.
Digest bytes	32 or 48 bytes	Digest value	SHA-256-width digest for P-256; SHA-384-width digest for P-384.

Table 82: ECC Dynamic Authentication Examples

Profile	Alg	Key Ref	Tag 81	Response 82	Command APDU
ECC P-256	11	9A/9E	32 bytes	DER ECDSA signature, about 70-72 bytes	44-byte short APDU.
ECC P-384	14	9A/9E	48 bytes	DER ECDSA signature, about 102-104 bytes	60-byte short APDU.

C.4.3 Worked Dynamic Authentication Examples

Worked examples are generated from
test-vectors/piv-auto-demo/generated/piv-auto-simulation-report.json.

Table 83: PIV Auto Worked Example: PIV Authentication RSA-1024

Field	Value
Profile	9a-rsa1024
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9a-rsa1024. Informative legacy/non-current test-only coverage.
Algorithm ID	0x06
Key Reference	0x9A
KMAC Challenge Digest	5B2FC08860E3999EEF8F6CF5D2B0C9AE05CBB28A5B423BFE4F646F6F47FCA4F2

Table 85: PIV Auto Worked Example: PIV Authentication RSA-2048

[illegible]

Table 86: PIV Auto Worked Example: Card Authentication RSA-2048

Field	Value
Profile	9e-rsa2048
Source	Committed RSA-2048 card-authentication key in test-vectors/piv-auto-demo/source-pl2, originally imported from GSA FICAM card-builder. NIST SD 33 / IR 8347 and generated-card coverage.
Algorithm ID	0x07
Key Reference	0x9E
KMAC Challenge Digest	0864C776F2374124D3E63F0B0B29FC1C5F0E8FF8BB1FA80E2723293B86A0158E
Tag 81 Challenge Input	0001FF FF FF FF FF FF FF FF FF FF FF FF FF 003031300D060960864801650304020105000420 0864C776F2374124D3E63F0B0B29FC1C5F0E8FF8BB1FA80E2723293B86A0158E
GENERAL AUTHENTICATE APDU	0087079E00010A7C8201068200818201000001FFFFFFFFFFFFFFFFFFFFFFFFFFFF FF FF FF FF FF FF FF FF FF FF FF 003031 300D0609608648016503040201050004200864C776F2374124D3E63F0B0B29FC 1C5F0E8FF8BB1FA80E2723293B86A0158E00
Dynamic Authentication Response	7C820104828201009BB1AA3875DB4E41107DD91CEAAC36FE4CE5F969E1F7B9B1 99DFAB38CFA1EE36FBF197CD2777C4A6CE082623C9F36A9DC2514FE54DD12934 AFCAE16DFE14FFCF0F58E0A0F46567444968F1DE78B496B4222DB913A87B850 139B1E2A730124C4ED169919F1EB5FA3E15617E89BB9CED8FA996A63E126C239 EB6D142A5746FBEC3DFA3B287F944EE45586BA5AEB64087BDA0661DF7254C42D 9768141F7243382D61CC43A8CDBE4ADA849E05F013D947DF7FA608C38ABA3841 B038AA69D77461B00053F59C4D34906BFBD470A277D0722AAECD83117DE906B FCD36A16EFB00B3EACD4414F48528CCA4BE57826B4312424A65DABAD6FA1ECA3 DAF55F6E6E1022C89000
Response Tag 82 Length	256 bytes

Table 87: PIV Auto Worked Example: PIV Authentication RSA-3072

[illegible]

Field	Value
Dynamic Authentication Response	7C4A824830460221009D25A44E56572983819019DF26A2720E462A11D2CB3CD86A9142DEF4BCC1271B022100D084982BB9D3E2D3E6B0965956C71DBA0FE44FE328C0A2BD72951DFB4D9296FE9000
Response Tag 82 Length	72 bytes

Table 90: PIV Auto Worked Example: Card Authentication ECC P-256

Field	Value
Profile	9e-ecp256
Source	Committed P-256 signing key in test-vectors/piv-auto-demo/source-p12, originally imported from GSA FICAM card-builder. NIST SD 33 / IR 8347 and generated-card coverage.
Algorithm ID	0x11
Key Reference	0x9E
KMAC Challenge Digest	965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D
Tag 81 Challenge Input	965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D
GENERAL AUTHENTICATE APDU	0087119E267C2482008120965AF088B02DF65E7C299ABF53A5AB2DCBB3C789458DE8D0C819B19A9266658D00
Dynamic Authentication Response	7C4A824830460221008B204FA48FAA27A57D6E90F985199F284D2A8E0D0F2E051BBFBCACBA824619B9022100C6E04880D322DE7D10823EC78C8EA4CCB69FE394F694F1D12C797FA7BCAD76959000
Response Tag 82 Length	72 bytes

Table 91: PIV Auto Worked Example: PIV Authentication ECC P-384

Field	Value
Profile	9a-ecp384
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9a-ecp384. Generated-card coverage.
Algorithm ID	0x14
Key Reference	0x9A
KMAC Challenge Digest	C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D28
Tag 81 Challenge Input	C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D28
GENERAL AUTHENTICATE APDU	0087149A367C3482008130C813683447986D513FC17CED5DB4D335BFBEAFB1AF9E5200C6B7B1C1E3535D700C2E05F9011717E153998832B7588D2800

Field	Value
Dynamic Authentication Response	7C6A82683066023100D88ABABB9893D88341CFA2F1AB6E3D35E4AF02F4F8D085B5A3DE63574DCF694741D869066B8E6297429120614DCC9CE40231009A3F203BB5EA98839008B0D25F98C98DE8E87622F61279B2F2B29343912999A36311102B310F9985844BCC248350377B9000
Response Tag 82 Length	104 bytes

Table 92: PIV Auto Worked Example: Card Authentication ECC P-384

Field	Value
Profile	9e-ecp384
Source	Committed YubiKey loader material in test-vectors/piv-auto-yubikey-material/9e-ecp384. Generated-card coverage.
Algorithm ID	0x14
Key Reference	0x9E
KMAC Challenge Digest	F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC689
Tag 81 Challenge Input	F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC689
GENERAL AUTHENTICATE APDU	0087149E367C3482008130F3480C6C1DAD008D3E14D0C815D381F01420E9D3402A175BED097C6949E4BA447FA0A11745CE4D054A95B10A9D5CC68900
Dynamic Authentication Response	7C6982673065023034C6B2659FBE26A78D2615F8C0EBF2E9674292D9DE3CE8A31D9DA297B5AE236AC886E9E9C288D42474564BF0C46AE40C023100CCBD59A047D73F9FD58F60218FBCB4F99B18EBD7EB70EFE5CF64D5EB1A65F1676A867E1B1E5EB4CF9F34B44BE905D29C9000
Response Tag 82 Length	103 bytes

C.5 APDUs

Table 93: PIV Auto APDU Reference: Card Preparation

Operation	Command APDU	Success Response	Purpose
Select PIV Application	00A404000BA000000308000010000100	9000	Selects the PIV application before reading objects or authenticating.
Get Discovery Object	00CB3FFF035C017E	7E...9000	Reads the Discovery Object so the PD can determine application and PIN/VCI policy.

Operation	Command APDU	Success Response	Purpose
Verify PIN (optional)	0020008008313233343536FFFF	9000	Verifies example PIN 123456 padded with FF; failure responses include 63CX and 6983.

Table 94: PIV Auto APDU Reference: GENERAL AUTHENTICATE Command

Field	Encoding	Source	Notes
CLA	00	ISO/IEC 7816-4	Use 0C when a credential secure channel is active.
INS	87	ISO/IEC 7816-4	GENERAL AUTHENTICATE instruction.
P1	Algorithm ID	PIV profile	05, 07, 11, or 14 for supported PIV Auto profiles; 06 appears only in informative RSA-1024 legacy examples.
P2	Key Reference	PIV profile	9A or 9E for PIV Auto.
Lc	Short or extended	Command length	Short APDUs use one length byte; extended APDUs use 00 followed by two length bytes.
Command data	7C...820081.....	Dynamic Authentication Template	Tag 82 is empty to request a response; tag 81 carries the exact challenge input.
Le	00	Response request	Requests the maximum response supported by the card interface.

Table 95: PIV Auto APDU Reference: GENERAL AUTHENTICATE Response

Case	Response APDU	PD Handling	OSDP Payload
Complete response	7C...82...9000	Strip only the ISO status word.	Carry the complete 7C Dynamic Authentication Template in osdp_CRAUTHR or osdp_PIVSTATUS.
Additional response bytes	61XX	Issue GET RESPONSE with Le XX.	Concatenate returned data before reporting the final 7C template.
GET RESPONSE command	00C00000XX	Retrieve remaining card response bytes.	Not exposed directly to the ACU.

Case	Response APDU	PD Handling	OSDP Payload
Credential error	69.. or 6A..	Preserve command failure detail.	Report via osdp_ PDERROR <u>PIVERROR</u> when available; do not treat as a valid signed response.

Table 96: PIV Auto APDU Reference: OPACITY / VCI Context

Operation	Command Shape	Key Ref	Relationship to PIV Auto
OPACITY establish	0087[27/2E]04[Lc]7C...81.. ..8200	04	Establishes VCI secure messaging and returns cryptogram/CVC evidence.
PIV Auto challenge	0087[alg] [9A/9E] [Lc]7C... 820081.....00	9A/9E	Performs the autonomous key challenge after card preparation and any required VCI setup.
Response handling	7C...82...9000	9A/9E	The PD reports the returned signed response bytes exactly, excluding only SW1/SW2.

D Deprecated osdp_GENAUTH

The osdp_GENAUTH command is deprecated and expected to be removed in a future revision of this specification. Using a PIV credential to decrypt or process arbitrary data is unsafe and unnecessary for credential verification; implementations **SHOULD** use osdp_CAUTH flows instead. Controllers and PDs **SHOULD** plan migrations away from osdp_GENAUTH.